Practical No.1

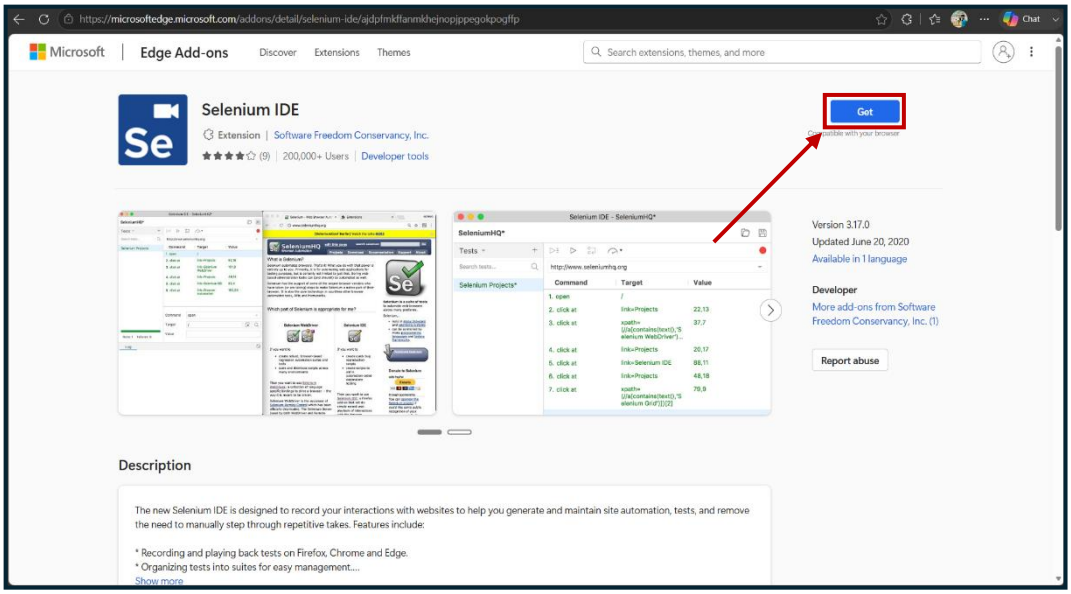
Selenium IDE: Multi-Format Validation Test Suite & WebDriver Export

Aim

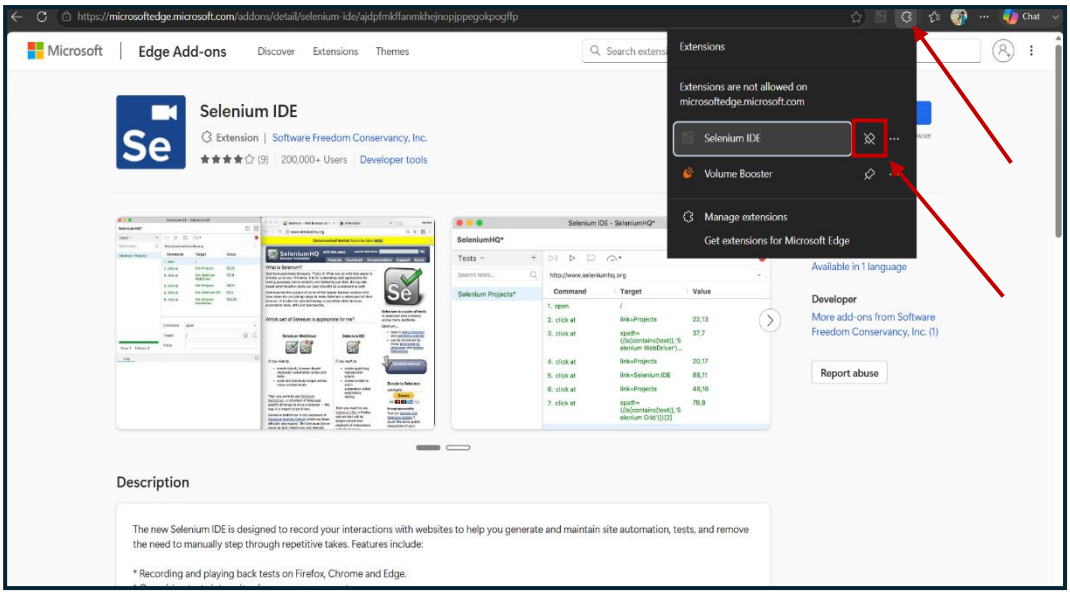
To install Selenium IDE and create a test suite containing a minimum of four test cases to validate different web page formats such as HTML page, XML response, JSON API output, and a dynamic form page. Execute the test suite and export the automation script in WebDriver format.

Steps:-

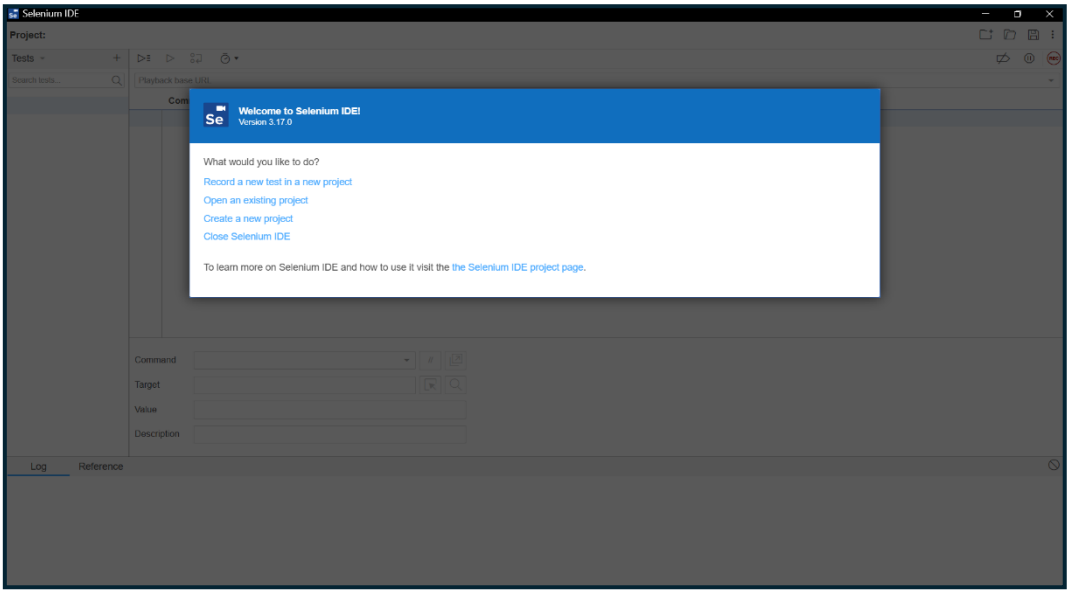
1. Open **Microsoft Edge** and Search **"Selenium IDE edge extension"** then Click on the first website. Click on **"Get"** to add extension in your browser.



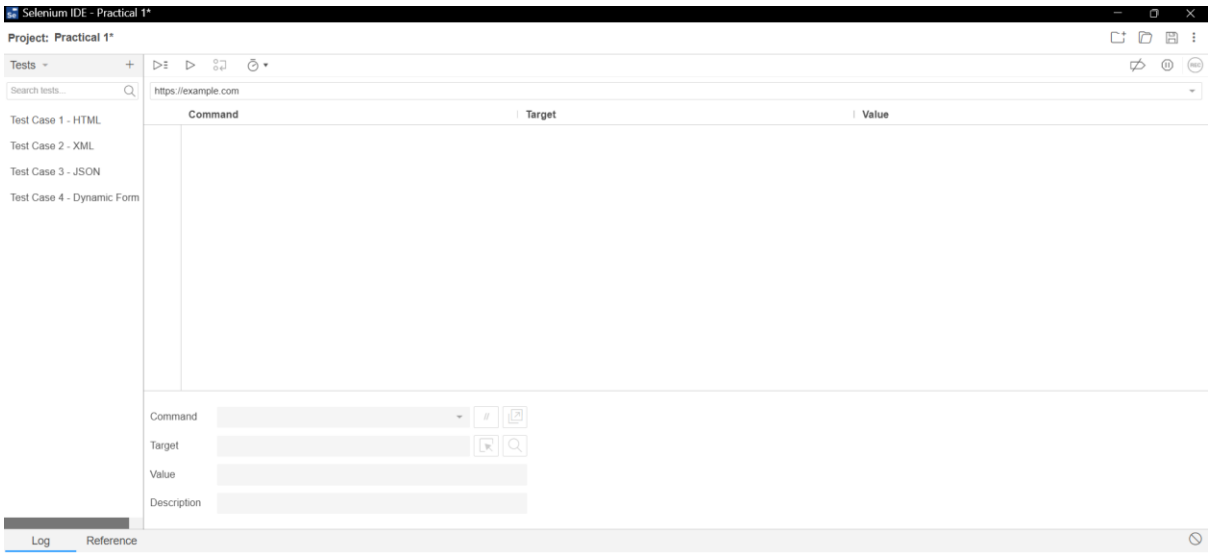
2. After adding extension click on extension icon and pin the Selenium IDE extension.



3. Click on Selenium IDE from extension to open Selenium IDE



4. Click on Create a new project and name it **"Practical 1"**
5. Create 4 test cases by clicking + icon next to the test in left panel
    - Test Case 1 – HTML
    - Test Case 2 – XML
    - Test Case 3 – JSON
    - Test Case 4 - Dynamic Form



Test Case 1 – HTML Page Validation

Base URL

https://example.com

Objective

To verify the contents and elements of a basic HTML webpage.

Selenium IDE Commands

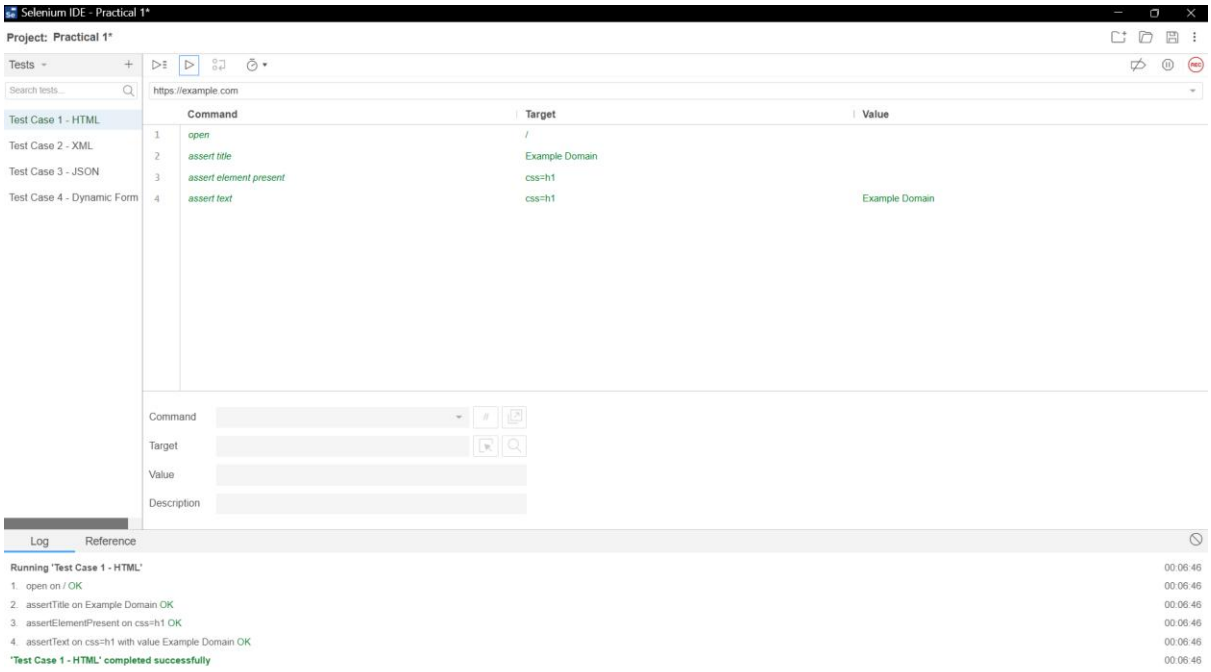
No.	Command	Target	Value
1	open	/	
2	assert title	Example Domain	
3	assert element present	css=h1	
4	assert text	css=h1	Example Domain

Expected Result

The HTML page should open successfully. The page title, heading element, and heading text should be verified successfully.

Result

Test Case 1 – HTML Page executed successfully.



Test Case 2 – XML Response Validation

URL

https://www.w3schools.com/xml/note.xml

Objective

To verify that the XML response is loaded correctly and contains the expected XML data.

Selenium IDE Commands

No.	Command	Target	Value
1	open	https://www.w3schools.com/xml/note.xml	
2	assert element present	css=note	
3	execute script	return document.documentElement.textContent.includes("Jani");	
4	execute script	return document.documentElement.textContent.includes("Tove");	
5	execute script	return document.documentElement.textContent.includes("Reminder");	

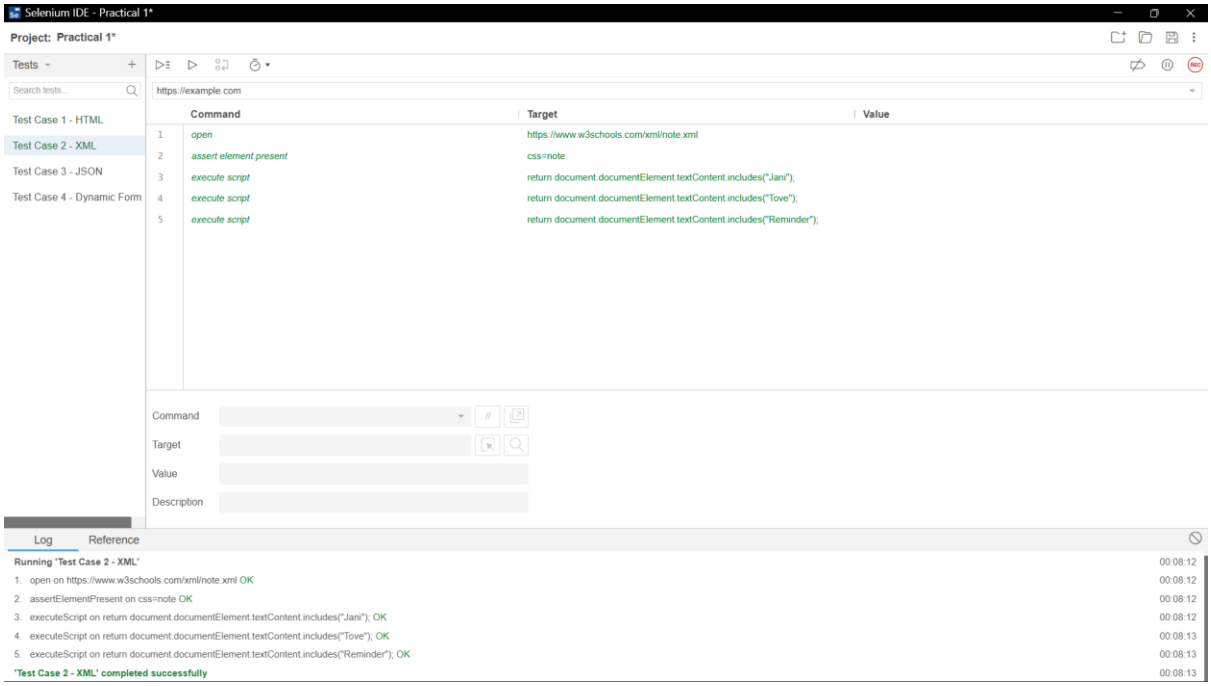
Expected Result

The XML response should load successfully and contain the values:

- Tove
- Jani
- Reminder

Result

Test Case 2 – XML Response executed successfully.



Test Case 3 – JSON API Validation

URL

https://jsonplaceholder.typicode.com/posts/1

Objective

To validate the JSON API response and verify important JSON fields.

Selenium IDE Commands

No.	Command	Target	Value
1	open	https://jsonplaceholder.typicode.com/posts/1	
2	execute script	const data = JSON.parse(document.body.innerText); return data.userId === 1;	
3	execute script	const data = JSON.parse(document.body.innerText); return data.id === 1;	
4	execute script	const data = JSON.parse(document.body.innerText); return data.title !== undefined && data.title !== "";	

Expected Result

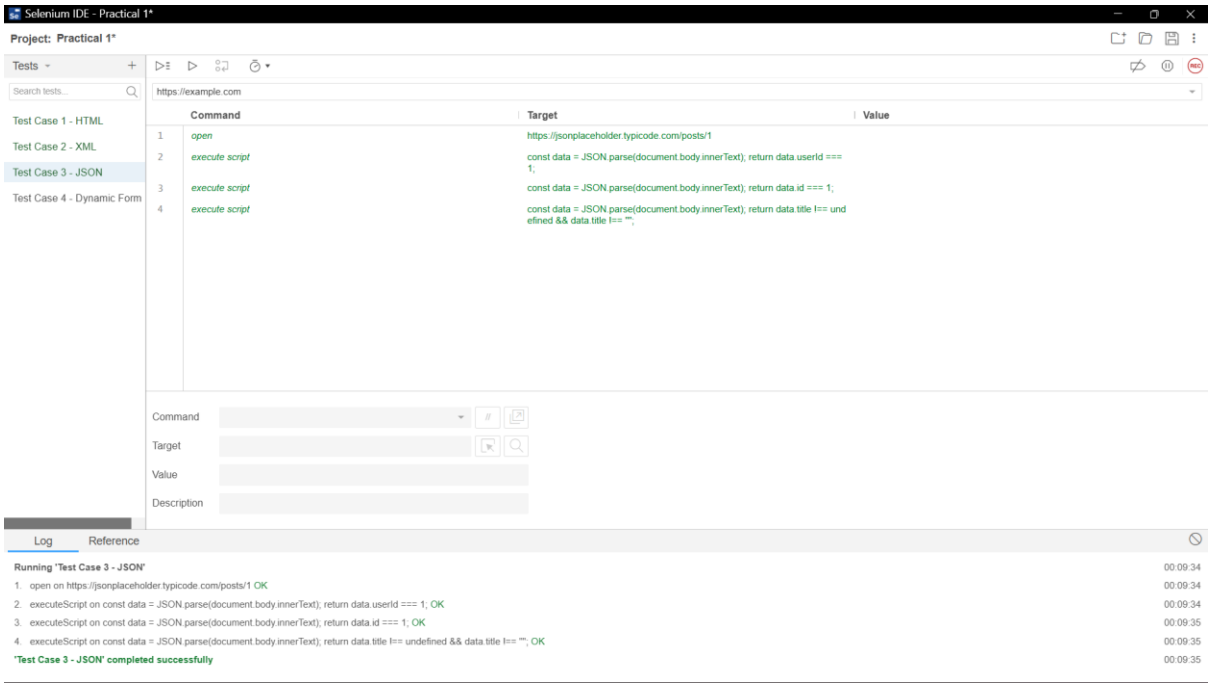
The JSON response should be successfully parsed and should contain:

- userId = 1

- id = 1
- A non-empty title

Result

Test Case 3 – JSON API executed successfully.



Test Case 4 – Dynamic Form Validation

URL

https://www.selenium.dev/selenium/web/web-form.html

Objective

To verify the functionality of a dynamic web form by entering data and submitting the form.

Selenium IDE Commands

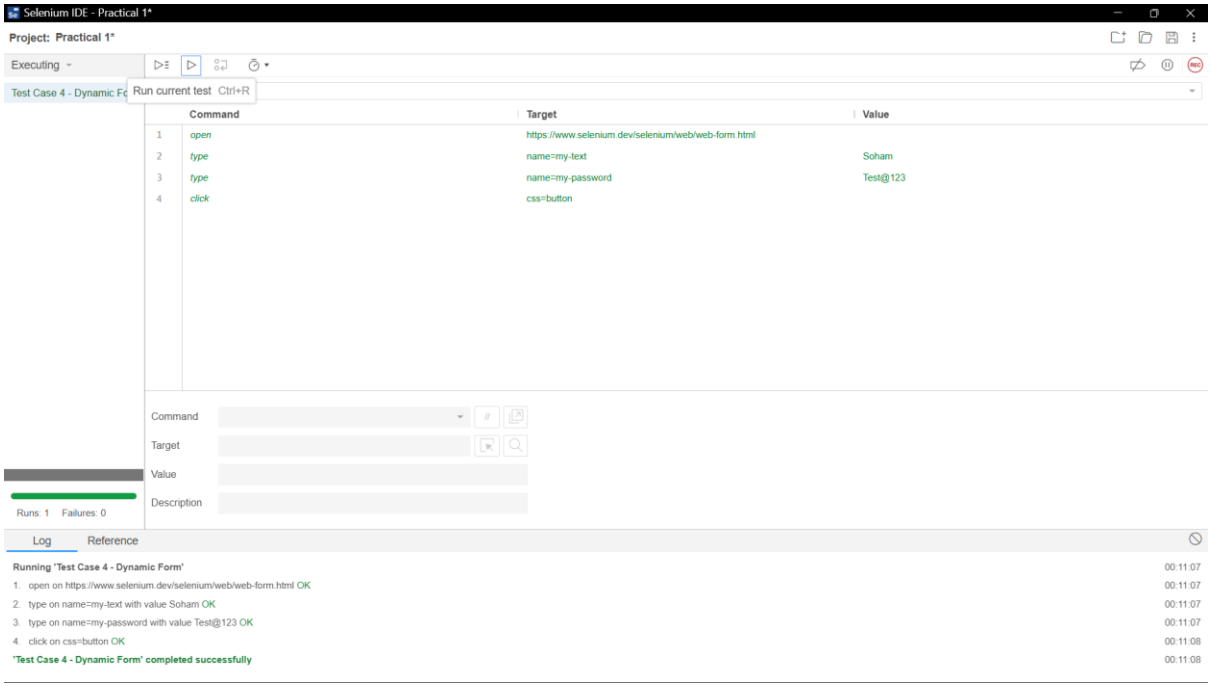
No.	Command	Target	Value
1	open	https://www.selenium.dev/selenium/web/web-form.html	
2	type	name=my-text	Soham
3	type	name=my-password	Test@123
4	click	css=button	

Expected Result

The form should accept the entered username/text and password, submit successfully, and display the message **Received!**

Result

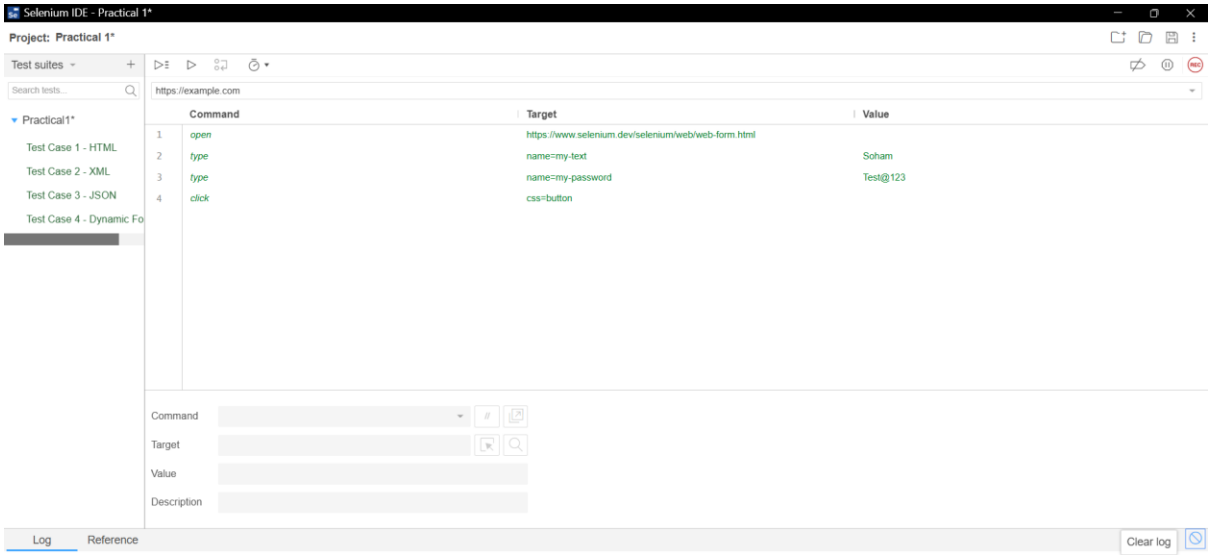
Test Case 4 – Dynamic Form executed successfully.



Test Suite Execution

After creating all four test cases:

1. Open the Selenium IDE project.
2. Select the test suite.
3. Rename Test Suite To **Practical1**
4. Right click on the test suite and add all 4 test we created
5. Verify that all four test cases are present.



---

6. Click **Run Suite**.
7. Observe the execution log.
8. Verify that each test case shows **Passed/OK**.
9. Take a screenshot of the successful execution for submission.

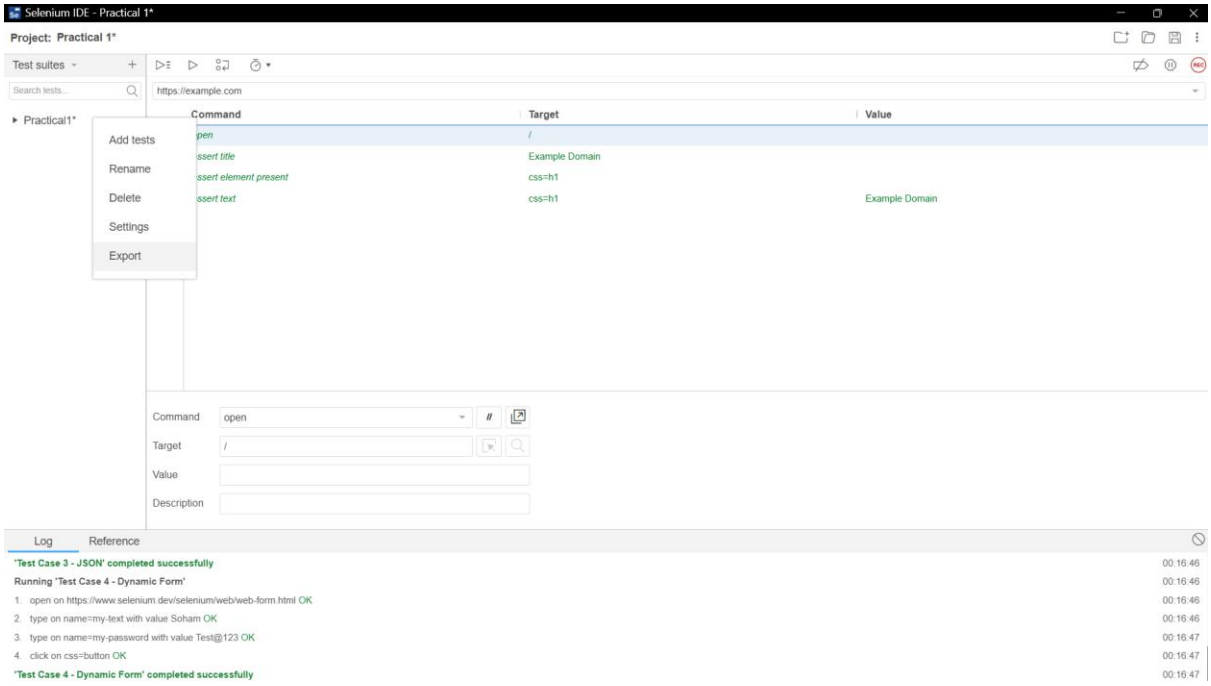
Test Execution Summary

Test Case	Format	Expected Result	Status
TC01	HTML	HTML elements validated	Pass
TC02	XML	XML content validated	Pass
TC03	JSON	JSON fields validated	Pass
TC04	Dynamic Form	Form submitted successfully	Pass

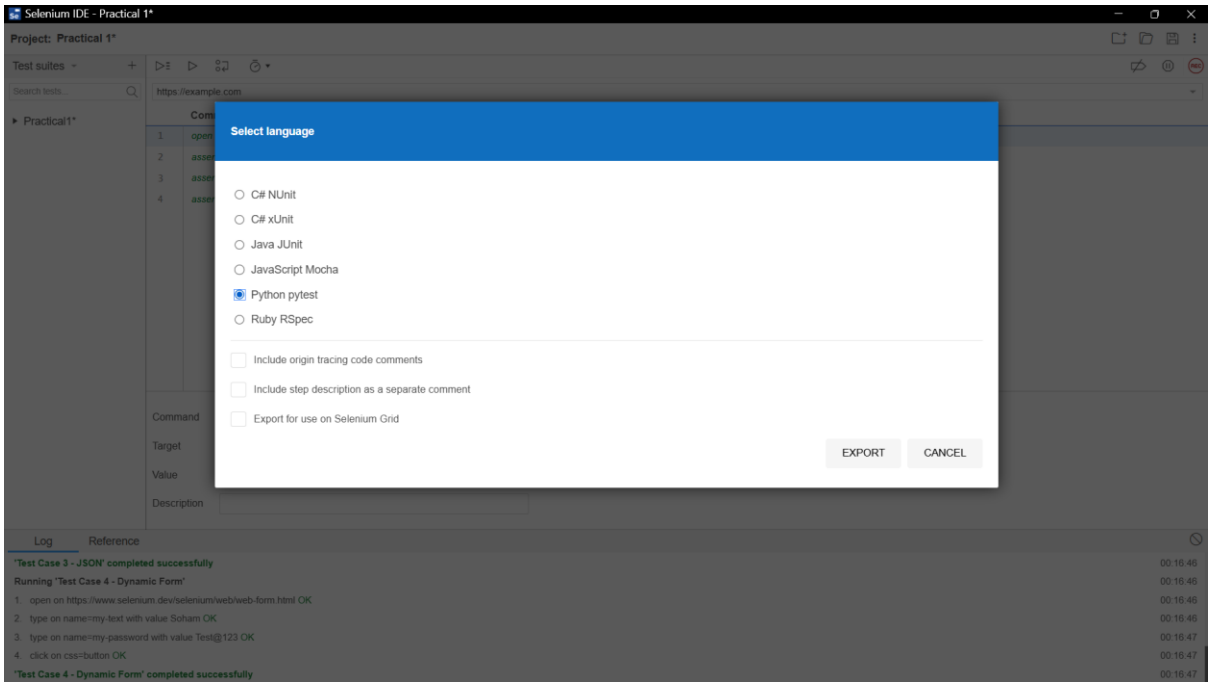
Export WebDriver Script

After successful execution:

1. Open the Selenium IDE project.
2. Select the required test case/project.
3. Open the **Export** option.



4. Select a WebDriver-compatible programming language such as **Java, Python, or JavaScript**.



5. Export the generated automation script.
6. Save the script.

Practical No. 2

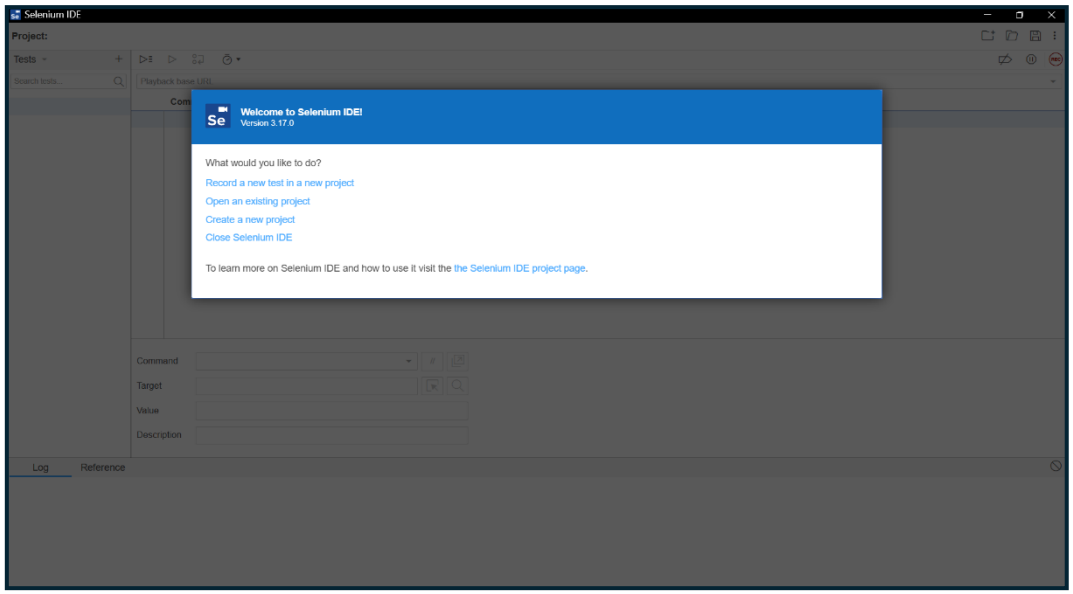
Selenium IDE: Cross-Website Functional Test Suite Execution & Verification

Aim

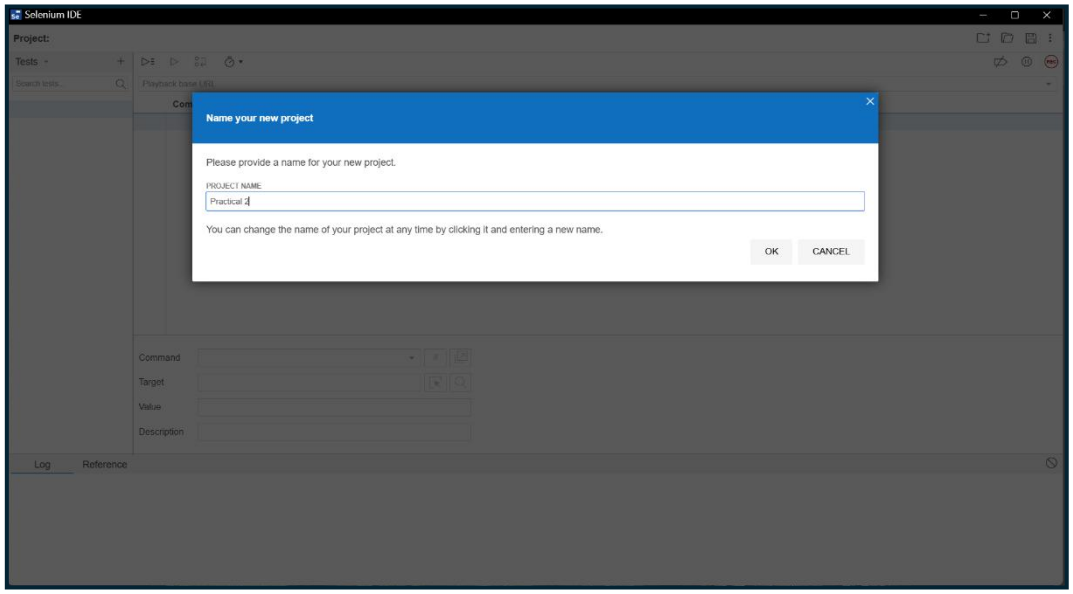
To develop and execute a cross-website functional test suite using **Selenium IDE** for two different websites. The test suite performs functional operations such as opening web pages, entering data into forms, clicking buttons, adding a product to the shopping cart, and verifying the displayed content.

Steps:-

1. **Click on the Selenium IDE extension to open Selenium IDE.**



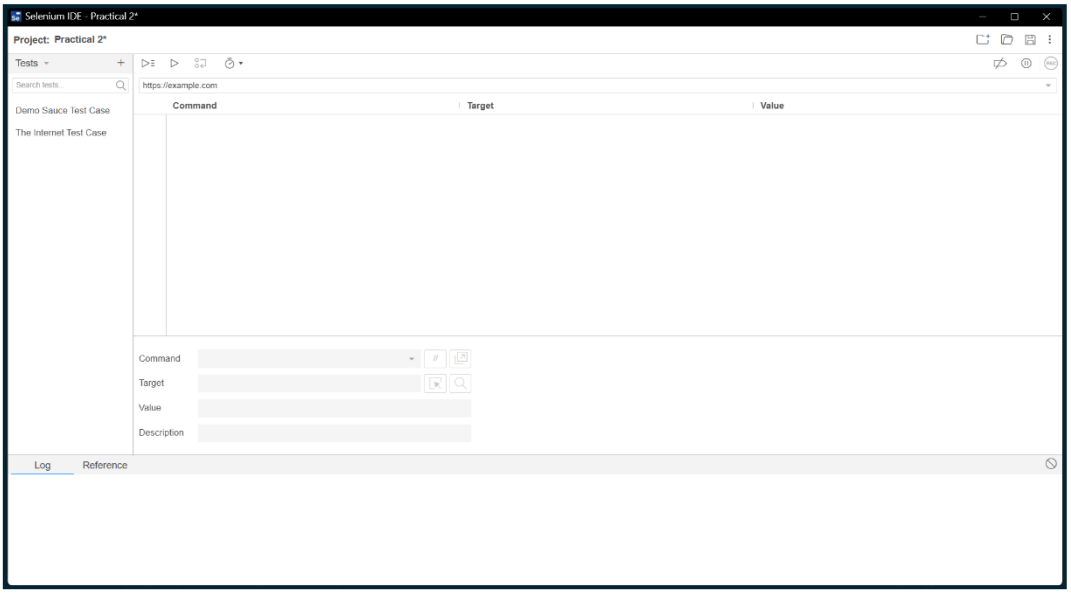
2. **Click on Create a new project and enter the project name: Practical 2**



3. **Create two test cases by clicking the + icon next to Tests.**

   The two test cases created are:

   - **Test Case 1 – Demo Sauce Test Case**
   - **Test Case 2 – The Internet Test Case**



Test Case 1 – Demo Sauce Test Case

Base URL

https://www.saucedemo.com

Objective

To verify the login functionality of the Sauce Demo website, add a product to the shopping cart, navigate to the cart, and verify that the selected product is displayed.

Test Data

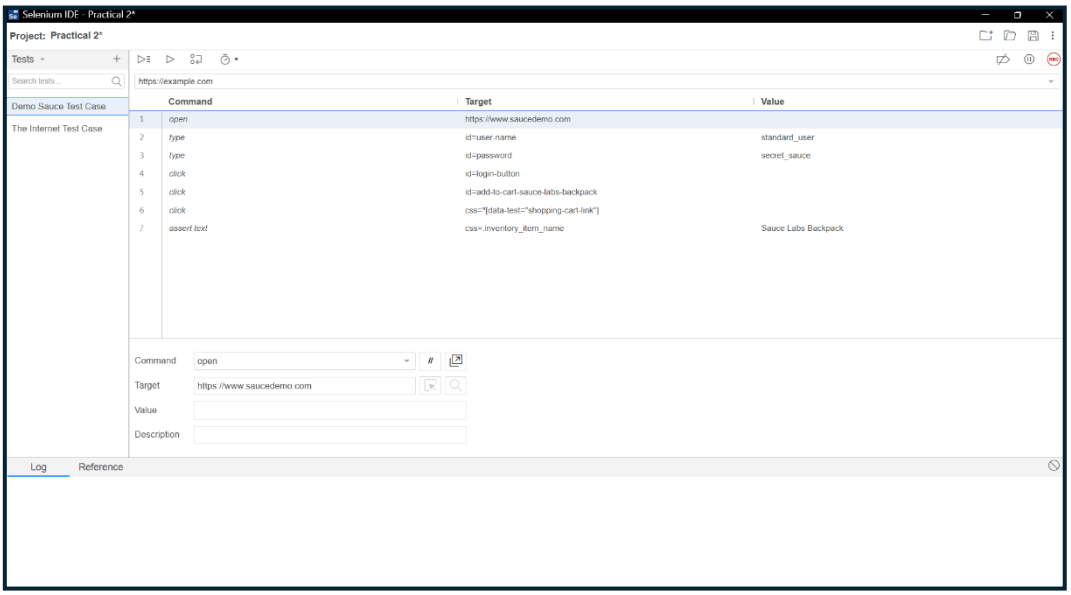
Username: standard_user

Password: secret_sauce

Product: Sauce Labs Backpack

Selenium IDE Commands

No.	Command	Target	Value
1	open	https://www.saucedemo.com	
2	type	id=user-name	standard_user
3	type	id=password	secret_sauce
4	click	id=login-button	
5	click	id=add-to-cart-sauce-labs-backpack	
6	click	css=*[data-test="shopping-cart-link"]	
7	assert text	css=.inventory_item_name	Sauce Labs Backpack



Description of Commands

Step 1 – Open

The Sauce Demo website is opened using the open command.

Step 2 – Enter Username

The username standard_user is entered into the username field.

Step 3 – Enter Password

The password secret_sauce is entered into the password field.

Step 4 – Login

The Login button is clicked using the click command.

Step 5 – Add Product

The **Sauce Labs Backpack** is added to the shopping cart.

Step 6 – Open Shopping Cart

The shopping cart icon is clicked to open the cart page.

Step 7 – Verify Product

The assert text command is used to verify that **Sauce Labs Backpack** is displayed in the shopping cart.

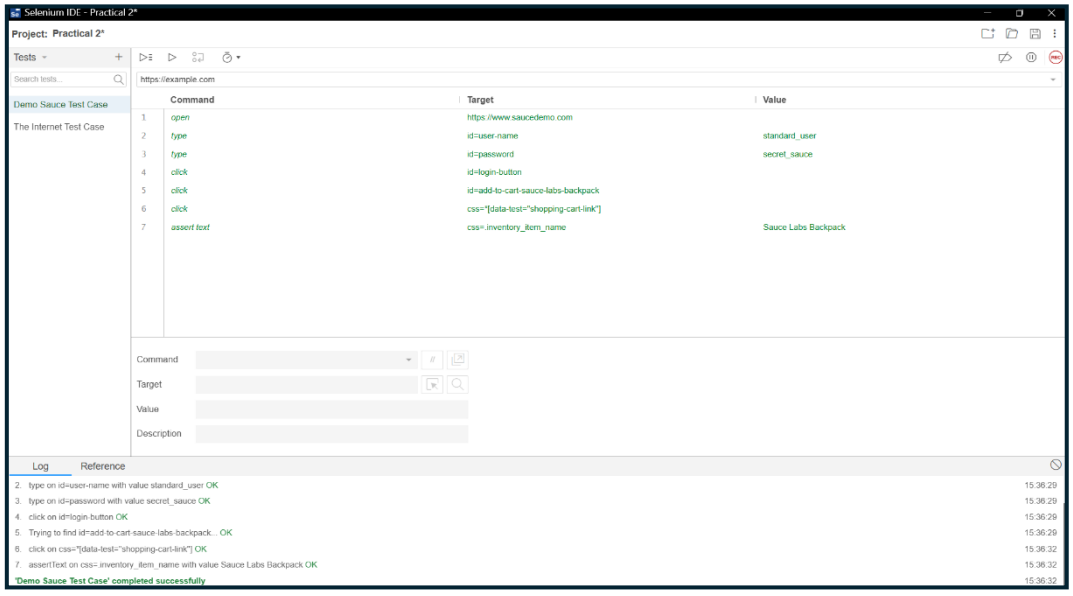
Expected Result

The Sauce Demo website should open successfully. The user should be able to log in using valid credentials. After successful login, the user should be able to add **Sauce Labs Backpack** to the shopping cart and open the cart. The selected product should be displayed in the shopping cart.

Result

Test Case 1 – Demo Sauce Test Case executed successfully.

Status: PASS



Test Case 2 – The Internet Test Case

Base URL

https://the-internet.herokuapp.com

Objective

To verify the login functionality of The Internet website by entering valid username and password and clicking the Login button.

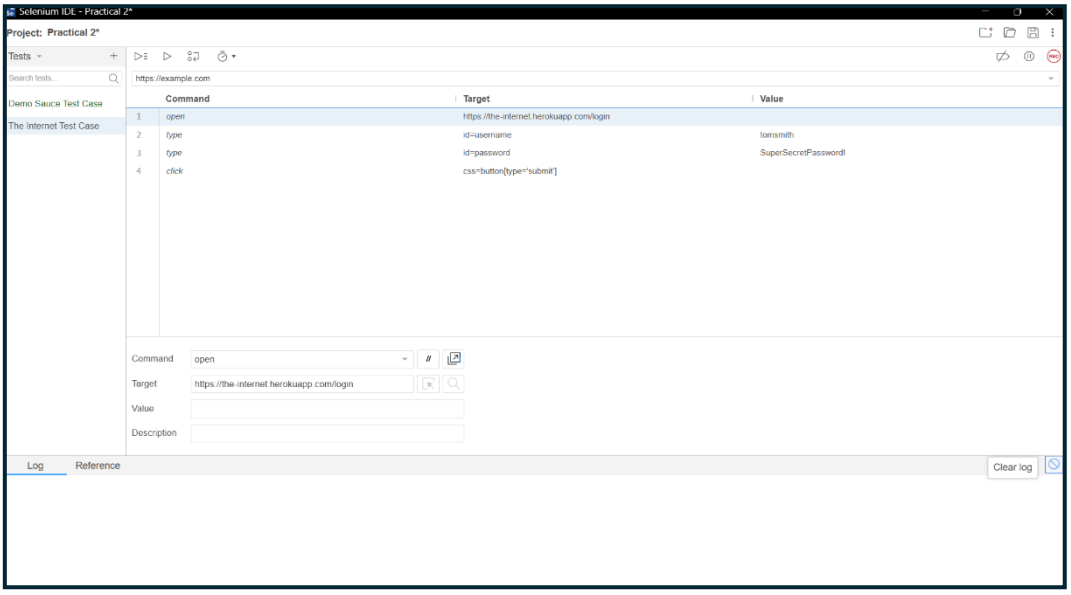
Test Data

Username: tomsmith

Password: SuperSecretPassword!

Selenium IDE Commands

No.	Command	Target	Value
1	open	https://the-internet.herokuapp.com/login	
2	type	id=username	tomsmith
3	type	id=password	SuperSecretPassword!
4	click	css=button[type='submit']	



Description of Commands

Step 1 – Open Login Page

The login page of The Internet website is opened.

Step 2 – Enter Username

The username tomsmith is entered into the username field.

Step 3 – Enter Password

The password SuperSecretPassword! is entered into the password field.

Step 4 – Click Login

The Login button is clicked to submit the form.

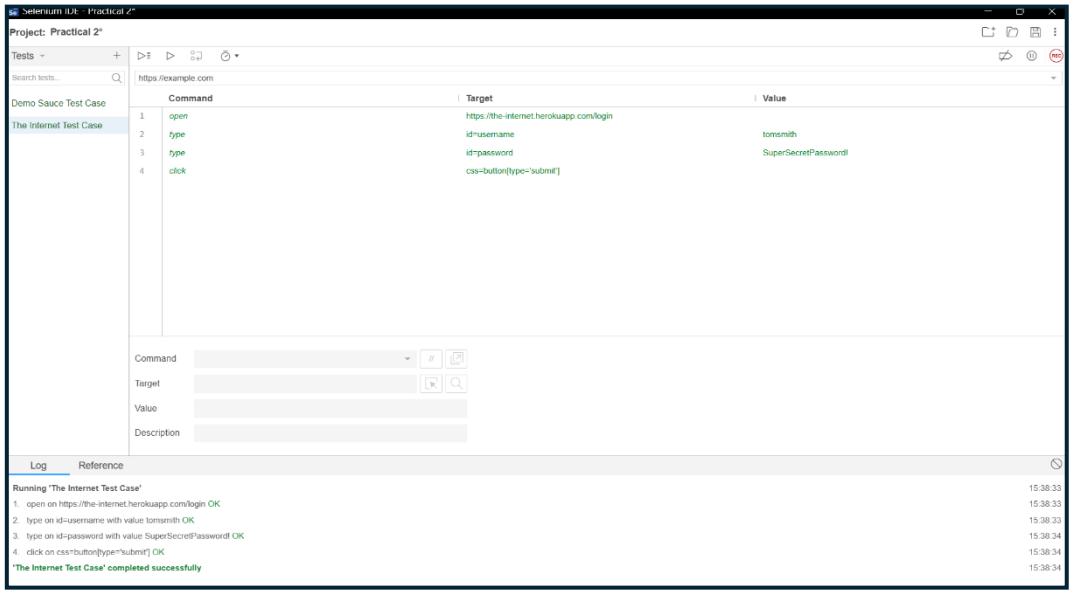
Expected Result

The login form should accept the valid username and password. After clicking the Login button, the user should be successfully redirected to the secure area of the website.

Result

Test Case 2 – The Internet Test Case executed successfully.

Status: PASS



Practical No. 3

STLC Implementation: Test Plan, Manual Execution, Test Report & Defect Tracking

Aim

To prepare a **Test Plan, Test Scenarios, and Test Cases** for a given web application using **MS Excel / Google Sheets**, manually execute the test cases, compare the actual results with the expected results, and prepare a **Test Execution Report and Defect Report** according to the **Software Testing Life Cycle (STLC)** process.

Tool Used

MS Excel / Google Sheets

Application Under Test

Application Name: Sauce Demo

Application Type: E-commerce Web Application

URL: https://www.saucedemo.com/

Introduction

Manual testing is a software testing technique in which test cases are executed by a tester manually without using automation tools or scripts. The tester interacts with the application, enters test data, performs different operations, observes the output, and compares the actual result with the expected result.

In this practical, the **Sauce Demo** web application is selected for manual functional testing. Different functionalities such as login, product display, adding products to the cart, removing products, checkout, order completion, and logout are tested.

The test documentation is prepared using **MS Excel / Google Sheets**. The testing process follows the major phases of the **Software Testing Life Cycle (STLC)**.

Steps:-

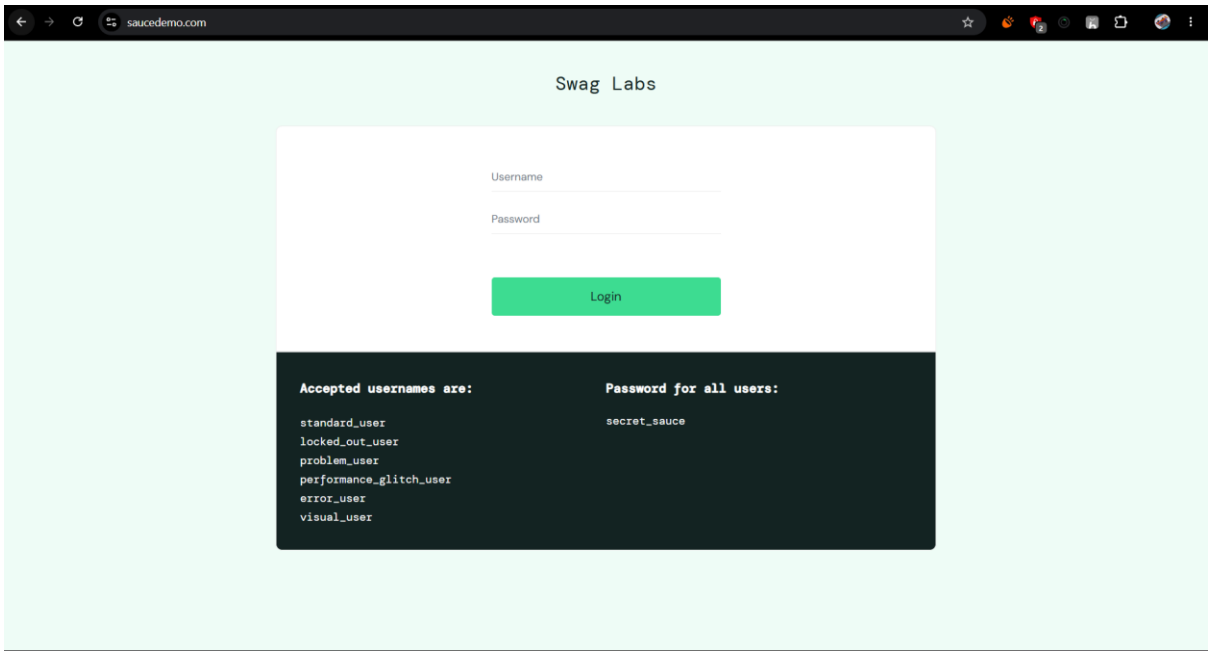
1. Open Microsoft Edge / Google Chrome

Open a web browser and enter the following URL:

https://www.saucedemo.com/

The Sauce Demo login page will be displayed.

Screenshot 1 – Application Login Page



Take a screenshot showing the Sauce Demo login page with the username, password, and Login button.

Caption:

Figure 1: Sauce Demo Login Page

2. Open Microsoft Excel / Google Sheets

Create a new workbook for manual testing documentation.

Save the file as:

Manual_Testing_Sauce_Demo.xlsx

Create the following sheets:

1. **Test Plan**
2. **Test Scenarios**
3. **Test Cases**
4. **Test Execution Report**
5. **Defect Report**

1. Test Plan

Create the first sheet and name it:

Test Plan

The test plan defines the testing objectives, scope, testing environment, resources, and testing approach.

Field	**Details**
Project Name	Sauce Demo Web Application
Test Plan ID	TP-001
Application Name	Sauce Demo
Application Type	E-commerce Web Application
Testing Type	Manual Functional Testing
Testing Approach	Black-Box Testing
Testing Level	System Testing
Testing Method	Manual Testing
Tool	MS Excel / Google Sheets
Browser	Microsoft Edge
Operating System	Windows
Tester	Soham

Test Environment	Web Browser with Internet Connection
Priority	High / Medium / Low

Test Objective

The objective is to verify the major functional features of the Sauce Demo web application and ensure that the application performs according to the expected requirements.

Scope of Testing

The following functionalities are included in testing:

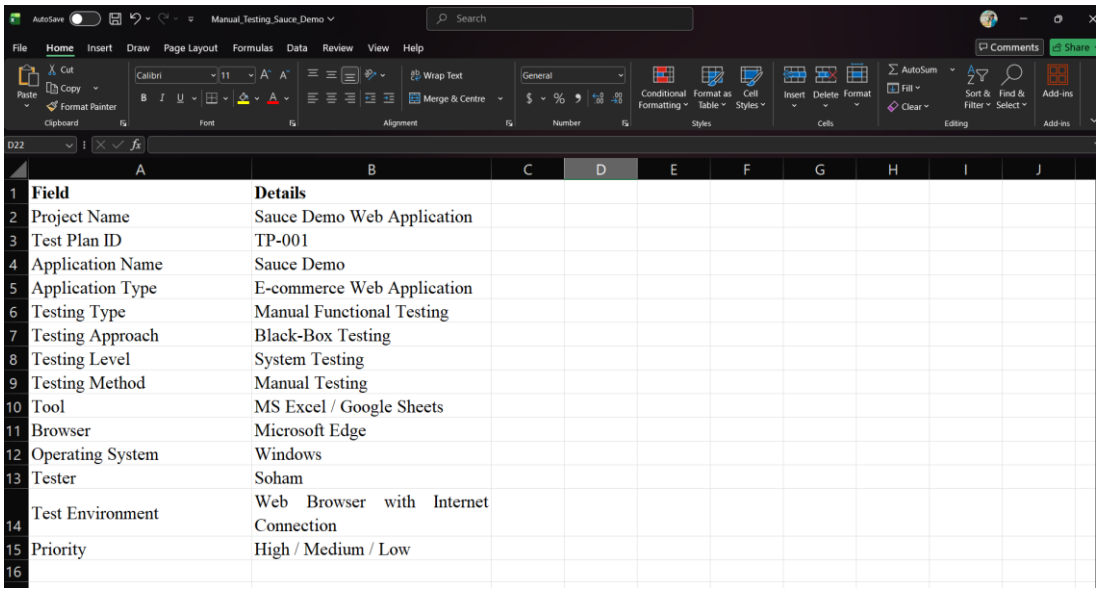
- Website accessibility
- User login
- Invalid login
- Product display
- Add product to cart
- Remove product from cart
- Shopping cart
- Checkout information
- Order completion
- Logout

Out of Scope

The following testing types are not included:

- Performance testing
- Load testing
- Stress testing
- Security testing
- API testing
- Database testing

Screenshot 2 – Test Plan Sheet



	A	B
1	**Field**	**Details**
2	Project Name	Sauce Demo Web Application
3	Test Plan ID	TP-001
4	Application Name	Sauce Demo
5	Application Type	E-commerce Web Application
6	Testing Type	Manual Functional Testing
7	Testing Approach	Black-Box Testing
8	Testing Level	System Testing
9	Testing Method	Manual Testing
10	Tool	MS Excel / Google Sheets
11	Browser	Microsoft Edge
12	Operating System	Windows
13	Tester	Soham
14	Test Environment	Web Browser with Internet Connection
15	Priority	High / Medium / Low
16		

Take a screenshot of your Excel **Test Plan** sheet showing the completed table.

Caption:

Figure 2: Test Plan Prepared in MS Excel

2. Test Environment

The following environment is used for manual execution.

Parameter	**Details**
Operating System	Windows
Browser	Microsoft Edge
Application	Sauce Demo
URL	https://www.saucedemo.com/
Testing Type	Manual Testing
Documentation Tool	MS Excel
Internet Connection	Available

3. Test Data

The following test data is used during manual execution.

Data	**Value**
Valid Username	standard_user

Valid Password	secret_sauce
Invalid Username	invalid_user
Invalid Password	invalid_password
Product	Sauce Labs Backpack
First Name	Soham
Last Name	Ghadge
Postal Code	411001

4. Test Scenarios

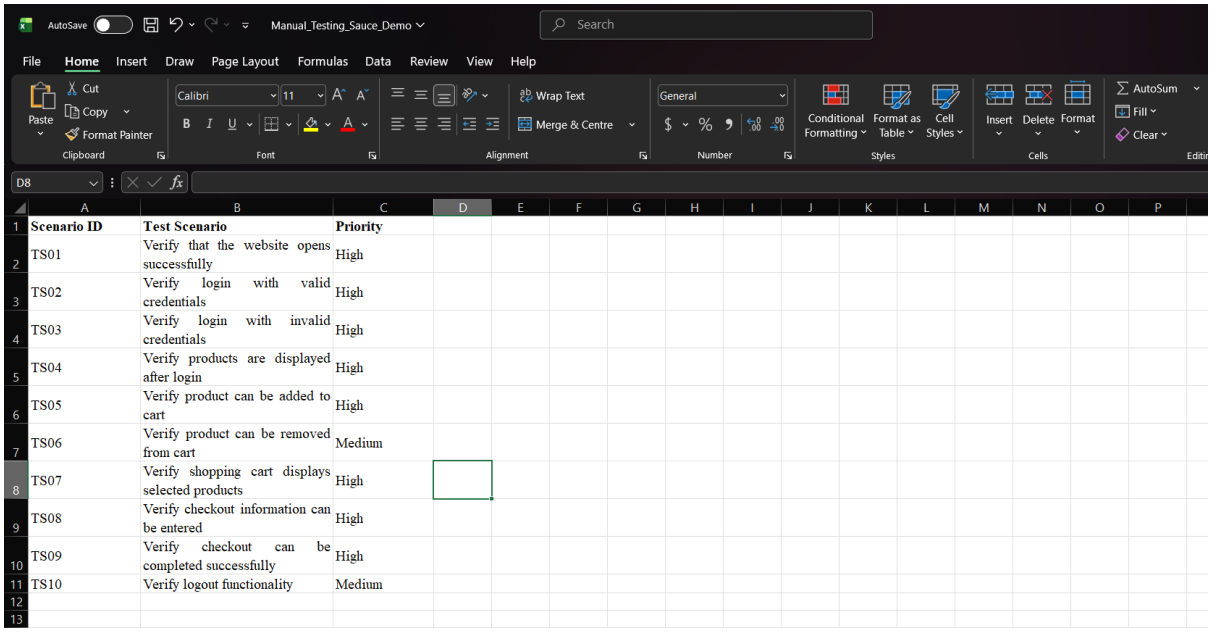
Create the second sheet and name it:

Test Scenarios

Test scenarios describe the high-level functionality that needs to be tested.

Scenario ID	Test Scenario	Priority
TS01	Verify that the website opens successfully	High
TS02	Verify login with valid credentials	High
TS03	Verify login with invalid credentials	High
TS04	Verify products are displayed after login	High
TS05	Verify product can be added to cart	High
TS06	Verify product can be removed from cart	Medium
TS07	Verify shopping cart displays selected products	High
TS08	Verify checkout information can be entered	High
TS09	Verify checkout can be completed successfully	High
TS10	Verify logout functionality	Medium

Screenshot 3 – Test Scenarios Sheet



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	**Scenario ID**	**Test Scenario**	**Priority**													
2	TS01	Verify that the website opens successfully	High													
3	TS02	Verify login with valid credentials	High													
4	TS03	Verify login with invalid credentials	High													
5	TS04	Verify products are displayed after login	High													
6	TS05	Verify product can be added to cart	High													
7	TS06	Verify product can be removed from cart	Medium													
8	TS07	Verify shopping cart displays selected products	High													
9	TS08	Verify checkout information can be entered	High													
10	TS09	Verify checkout can be completed successfully	High													
11	TS10	Verify logout functionality	Medium													
12																
13																

Take a screenshot showing the complete **Test Scenarios** sheet in Excel.

Caption:

Figure 3: Test Scenarios Prepared for Sauce Demo

5. Test Cases

Create the third sheet and name it:

Test Cases

Test cases provide detailed steps for testing each functionality.

Test Case 1 – Website Accessibility

Test Case ID: TC01

Scenario ID: TS01

Objective

To verify that the Sauce Demo website opens successfully.

Field	**Details**
Test Case ID	TC01
Scenario ID	TS01
Priority	High
Preconditions	Internet connection is available
Test Data	Sauce Demo URL

Test Steps

Step	Test Action	Expected Result
1	Open Microsoft Edge	Browser should open
2	Enter Sauce Demo URL	Website should open
3	Observe the login page	Username, Password and Login fields should be displayed

Expected Result

The Sauce Demo login page should open successfully.

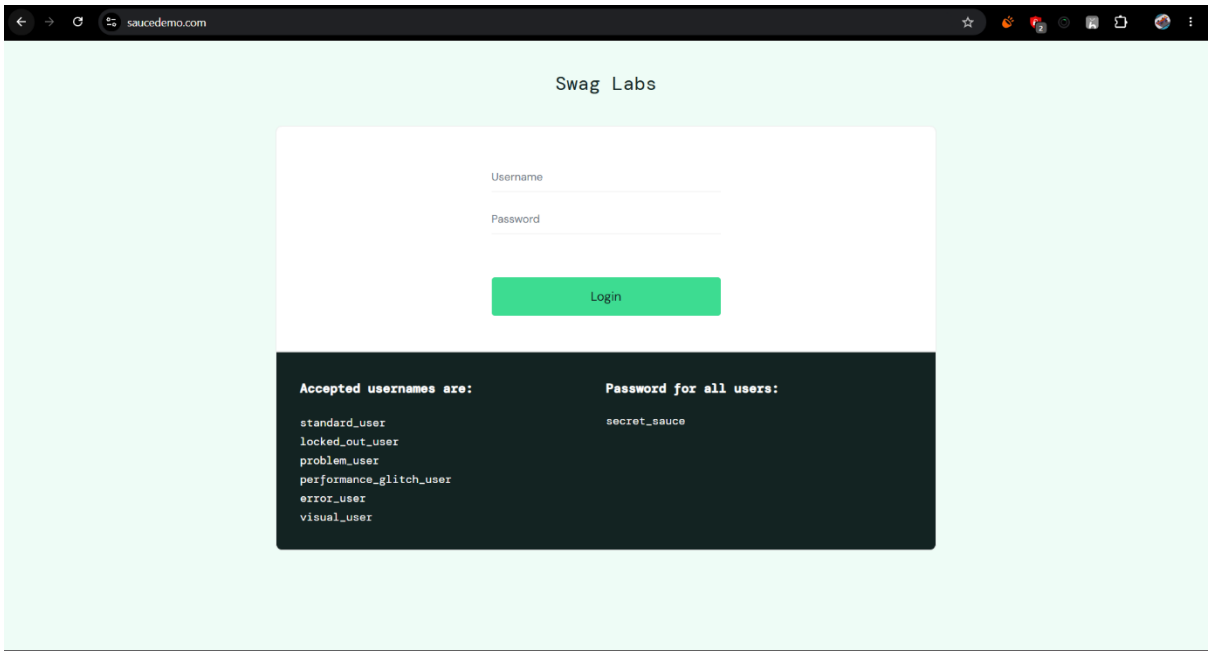
Actual Result

The Sauce Demo login page opened successfully.

Status

PASS

Screenshot 4 – Website Accessibility



You can use the screenshot of the login page from Figure 1 if you don't want duplicate screenshots. If your teacher wants a screenshot for every test case, take another one.

Test Case 2 – Valid Login

Test Case ID: TC02

Scenario ID: TS02

Objective

To verify that the user can successfully log in using valid credentials.

Test Data

Username: standard_user

Password: secret_sauce

Test Steps

Step	Test Action	Expected Result
1	Open Sauce Demo	Login page should be displayed
2	Enter standard_user	Username should be accepted
3	Enter secret_sauce	Password should be accepted
4	Click Login	User should be redirected to Products page
5	Observe page	Products heading should be displayed

Expected Result

The user should successfully log in and the Products page should be displayed.

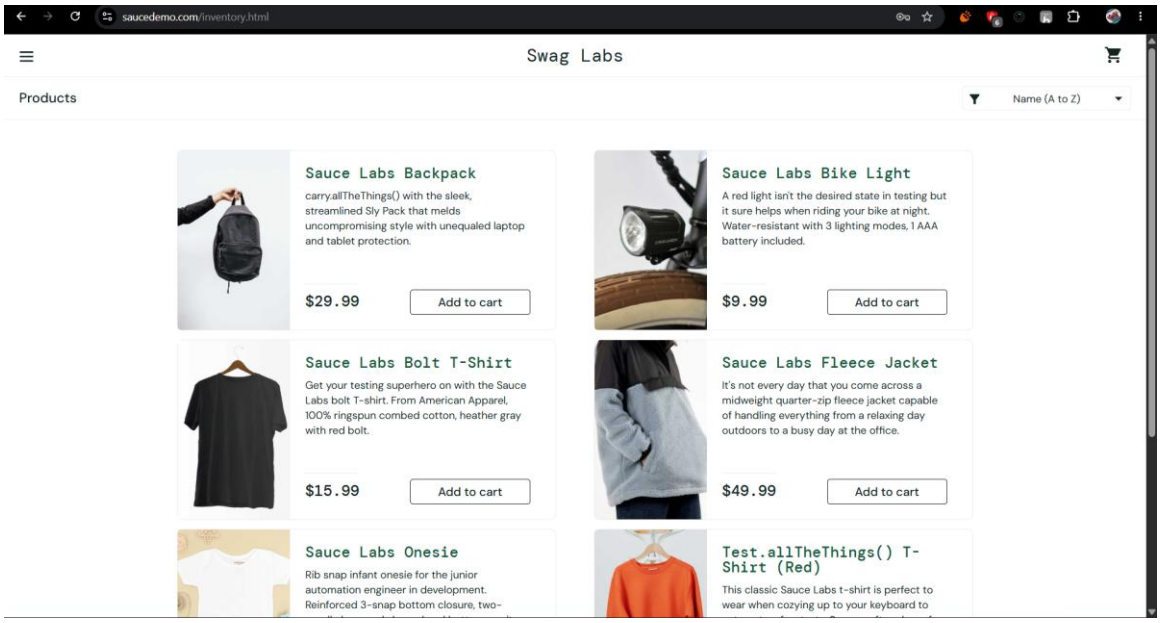
Actual Result

User successfully logged in and Products page was displayed.

Status

PASS

Screenshot 5 – Successful Login



Manually perform the login and take a screenshot of the **Products page after successful login**.

Caption:

Figure 5: Successful Login and Products Page

Test Case 3 – Invalid Login

Test Case ID: TC03

Scenario ID: TS03

Objective

To verify that the application does not allow login with invalid credentials.

Test Data

Username: invalid_user

Password: invalid_password

Test Steps

Step	Test Action	Expected Result
1	Open Sauce Demo	Login page should be displayed
2	Enter invalid username	Username should be accepted
3	Enter invalid password	Password should be accepted

4	Click Login	Login should not be successful
5	Observe error message	Error message should be displayed

Expected Result

The application should reject invalid credentials and display an appropriate error message.

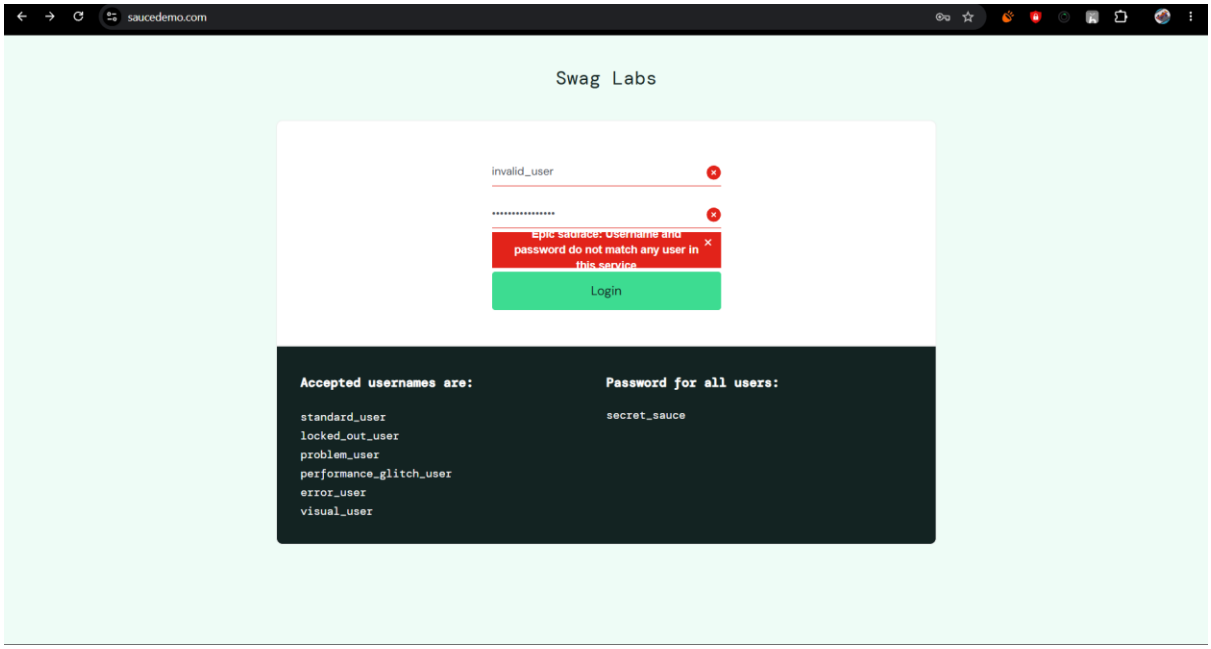
Actual Result

Invalid credentials were rejected and an error message was displayed.

Status

PASS

Screenshot 6 – Invalid Login Error



Enter the invalid credentials manually and take a screenshot showing the **error message**.

Caption:

Figure 6: Error Message for Invalid Login

Test Case 4 – Product Display

Test Case ID: TC04

Scenario ID: TS04

Objective

To verify that products are displayed after successful login.

Test Steps

Step	Test Action	Expected Result
1	Login with valid credentials	Products page should open
2	Observe product list	Products should be displayed
3	Check product names	Product names should be visible
4	Check product prices	Product prices should be visible

Expected Result

Products, product names, images, and prices should be displayed correctly.

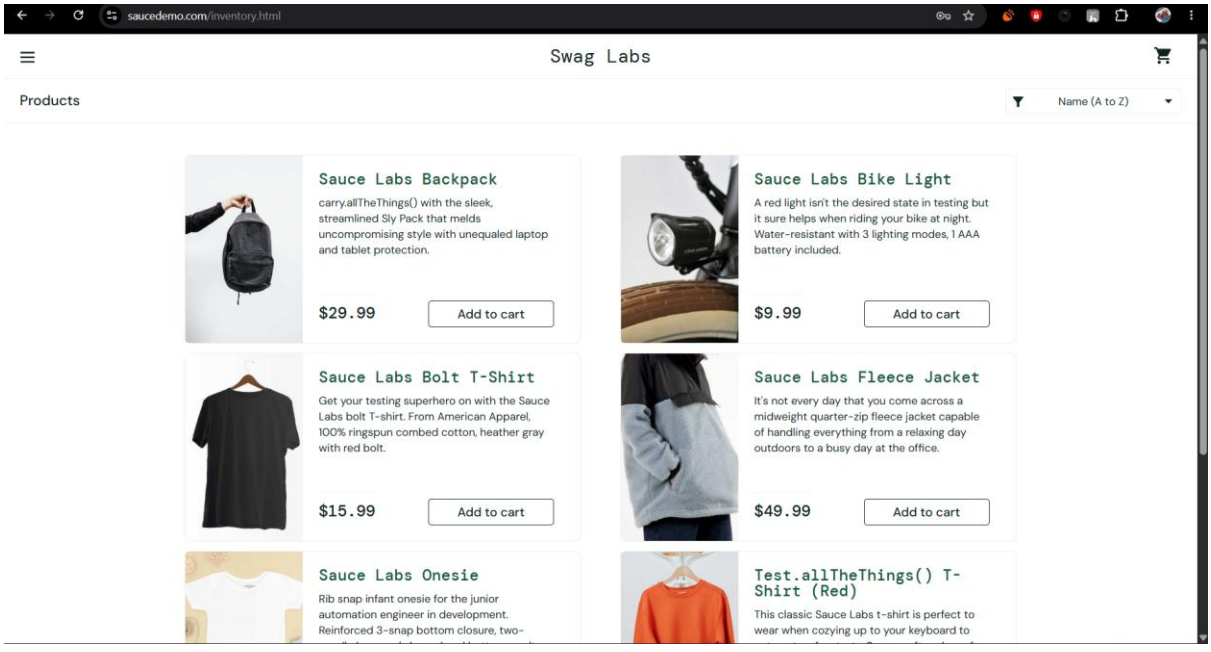
Actual Result

Products, product names, images, and prices were displayed correctly.

Status

PASS

Screenshot 7 – Product Page



Take a screenshot showing the Sauce Demo Products page.

Caption:

Figure 7: Products Displayed on Sauce Demo

Test Case 5 – Add Product to Cart

Test Case ID: TC05

Scenario ID: TS05

Objective

To verify that a product can be successfully added to the shopping cart.

Test Data

Product: **Sauce Labs Backpack**

Test Steps

Step	Test Action	Expected Result
1	Login successfully	Products page should open
2	Locate Sauce Labs Backpack	Product should be displayed
3	Click Add to Cart	Product should be added
4	Observe shopping cart	Cart count should be updated

Expected Result

The Sauce Labs Backpack should be added to the shopping cart.

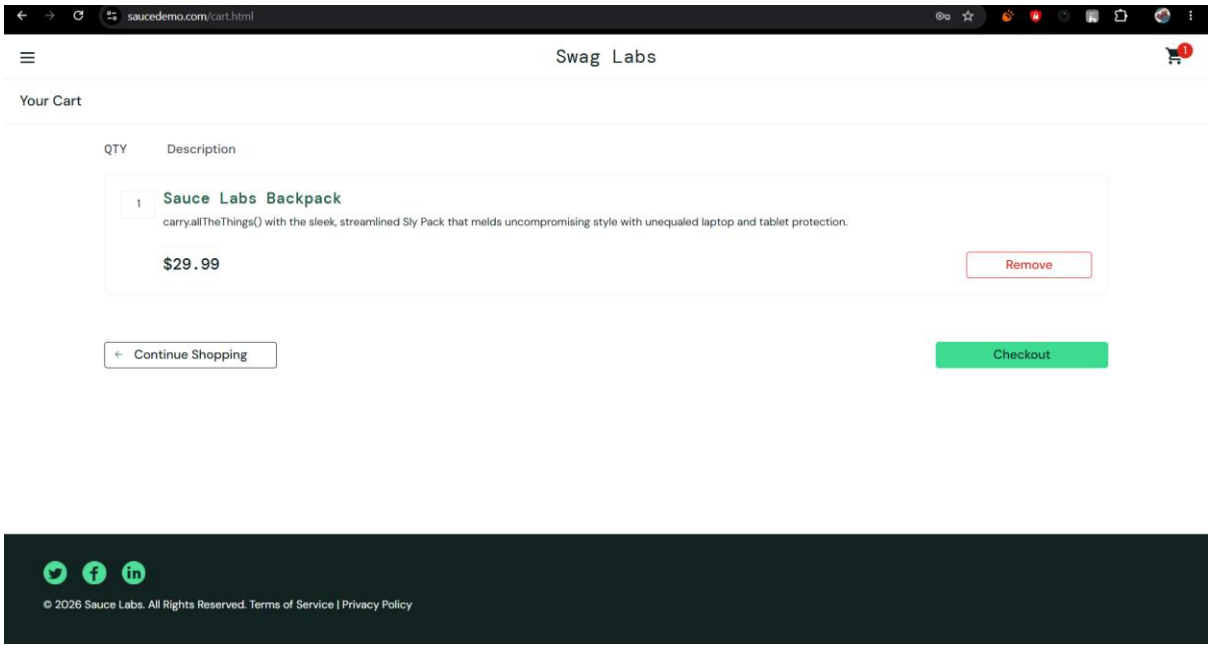
Actual Result

The Sauce Labs Backpack was successfully added to the shopping cart.

Status

PASS

Screenshot 8 – Product Added to Cart



Take a screenshot showing the product and the updated cart icon/count.

Caption:

Figure 8: Product Added to Shopping Cart

Test Case 6 – Remove Product from Cart

Test Case ID: TC06

Scenario ID: TS06

Objective

To verify that a selected product can be removed from the cart.

Test Steps

Step	Test Action	Expected Result
1	Login successfully	Products page should open
2	Add Sauce Labs Backpack	Product should be added
3	Click Remove	Product should be removed
4	Observe cart	Product should no longer be present

Expected Result

The selected product should be removed from the shopping cart.

Actual Result

The product was successfully removed from the shopping cart.

Status

PASS

Test Case 7 – Shopping Cart Verification

Test Case ID: TC07

Scenario ID: TS07

Objective

To verify that the selected product is displayed correctly in the shopping cart.

Test Steps

Step	Test Action	Expected Result
1	Login successfully	Products page should open
2	Add Sauce Labs Backpack	Product should be added
3	Click shopping cart	Cart page should open
4	Observe cart	Sauce Labs Backpack should be displayed
5	Verify quantity	Correct quantity should be displayed

Expected Result

The selected product should be displayed correctly in the shopping cart.

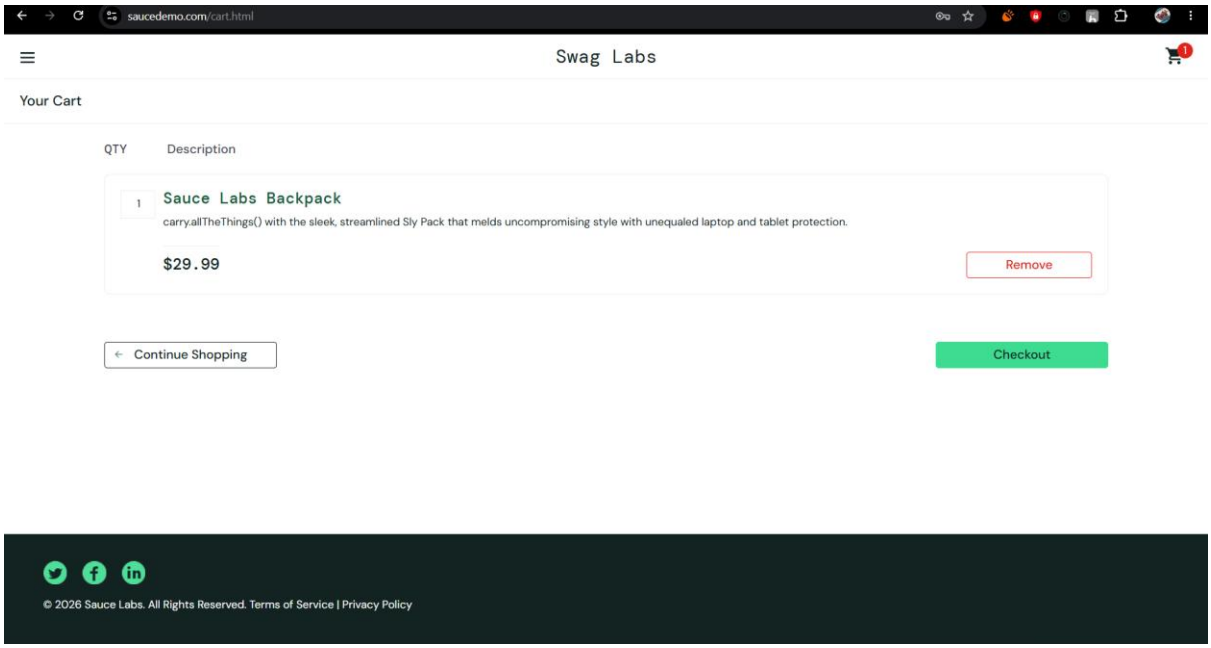
Actual Result

The selected product was displayed correctly in the shopping cart.

Status

PASS

Screenshot 9 – Shopping Cart



Open the cart and take a screenshot showing **Sauce Labs Backpack** inside the cart.

Caption:

Figure 9: Shopping Cart with Selected Product

Test Case 8 – Checkout Information

Test Case ID: TC08

Scenario ID: TS08

Objective

To verify that the checkout information form accepts valid customer information.

Test Data

First Name: Soham

Last Name: Ghadge

Postal Code: 411001

Test Steps

Step	Test Action	Expected Result
1	Add product to cart	Product should be added
2	Open shopping cart	Cart page should open

3	Click Checkout	Checkout page should open
4	Enter First Name	First name should be accepted
5	Enter Last Name	Last name should be accepted
6	Enter Postal Code	Postal code should be accepted
7	Click Continue	Checkout overview should be displayed

Expected Result

The checkout form should accept valid customer information and continue to the checkout overview.

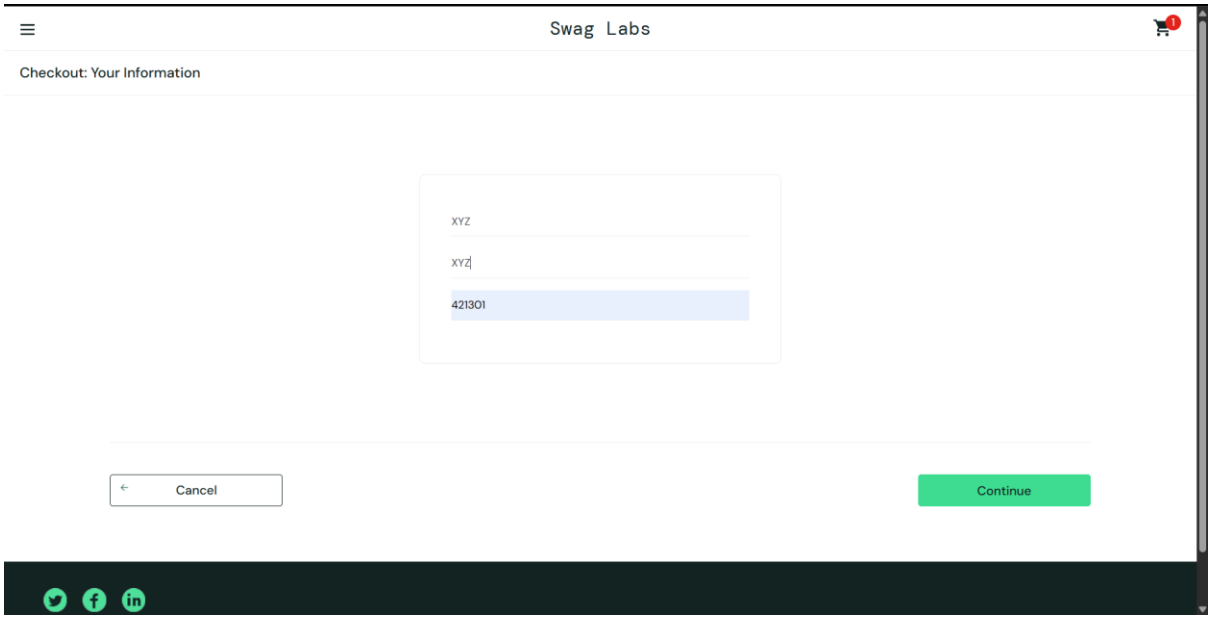
Actual Result

The checkout information was accepted and the checkout overview was displayed.

Status

PASS

Screenshot 10 – Checkout Information



Take a screenshot of the checkout form after entering the details.

Caption:

Figure 10: Checkout Information Form

Test Case 9 – Checkout Completion

Test Case ID: TC09

Scenario ID: TS09

Objective

To verify that the checkout process can be completed successfully.

Test Steps

Step	Test Action	Expected Result
1	Add product to cart	Product should be added
2	Open cart	Cart should open
3	Click Checkout	Checkout information page should open
4	Enter valid information	Information should be accepted
5	Click Continue	Checkout overview should open
6	Click Finish	Order should be completed
7	Observe confirmation	Order confirmation should be displayed

Expected Result

The order should be completed successfully and an order confirmation message should be displayed.

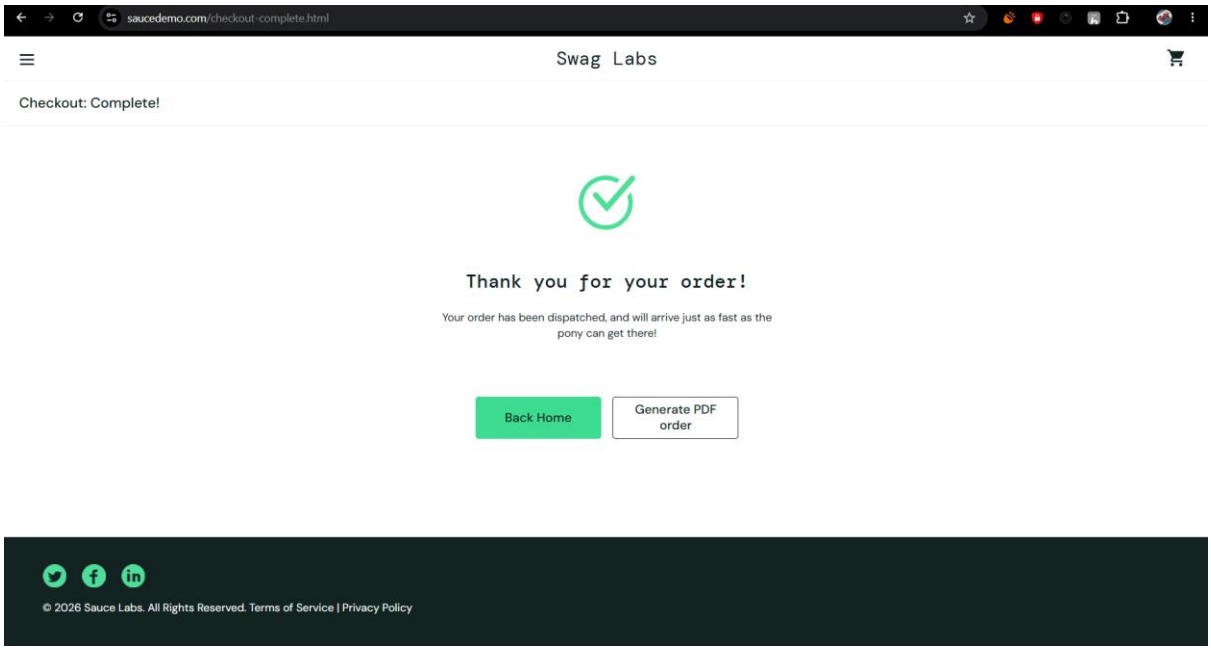
Actual Result

The order was completed successfully and the confirmation message was displayed.

Status

PASS

Screenshot 11 – Order Confirmation



Take a screenshot of the final order confirmation page.

Caption:

Figure 11: Successful Order Completion

Test Case 10 – Logout

Test Case ID: TC10

Scenario ID: TS10

Objective

To verify that the user can successfully log out from the application.

Test Steps

Step	Test Action	Expected Result
1	Login successfully	Products page should be displayed
2	Open the menu	Menu should be displayed
3	Click Logout	User should be logged out
4	Observe page	Login page should be displayed

Expected Result

The user should be logged out and redirected to the login page.

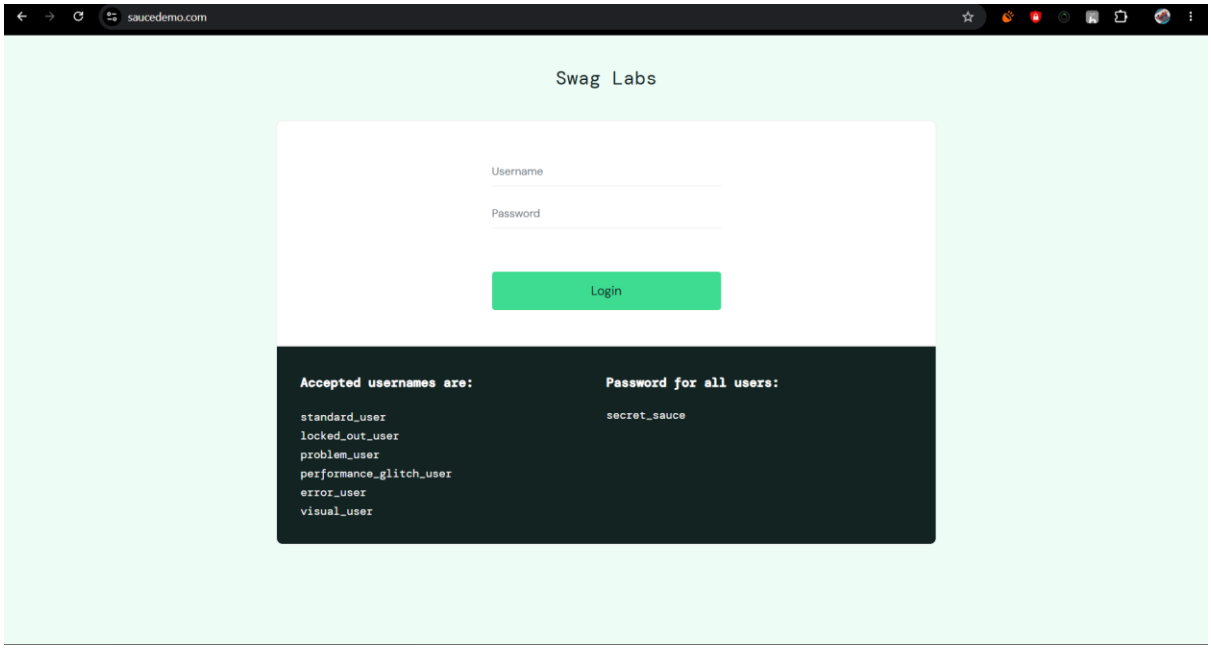
Actual Result

The user was successfully logged out and the login page was displayed.

Status

PASS

Screenshot 12 – Logout



Take a screenshot showing the login page after logout.

Caption:

Figure 12: Successful Logout

6. Test Execution Report

After manually executing all the test cases, create a new Excel sheet named:

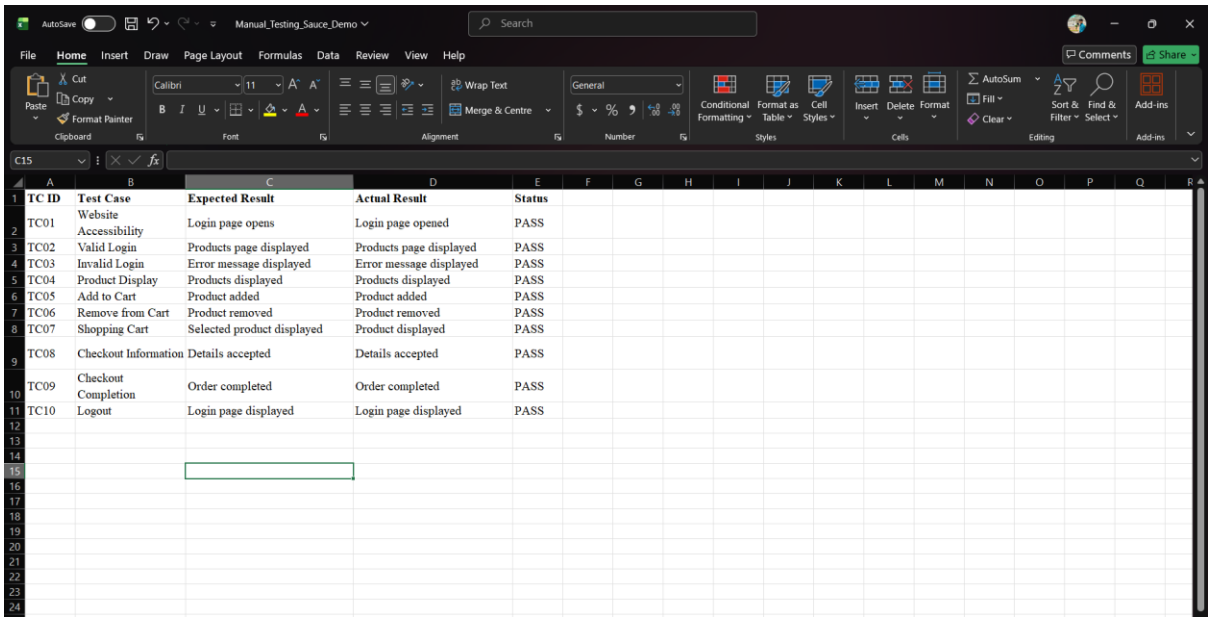
Test Execution Report

Record the actual results and status.

TC ID	Test Case	Expected Result	Actual Result	Status
TC01	Website Accessibility	Login page opens	Login page opened	PASS
TC02	Valid Login	Products page displayed	Products page displayed	PASS
TC03	Invalid Login	Error message displayed	Error message displayed	PASS

TC04	Product Display	Products displayed	Products displayed	PASS
TC05	Add to Cart	Product added	Product added	PASS
TC06	Remove from Cart	Product removed	Product removed	PASS
TC07	Shopping Cart	Selected product displayed	Product displayed	PASS
TC08	Checkout Information	Details accepted	Details accepted	PASS
TC09	Checkout Completion	Order completed	Order completed	PASS
TC10	Logout	Login page displayed	Login page displayed	PASS

Screenshot 13 – Test Execution Report



TC ID	Test Case	Expected Result	Actual Result	Status
TC01	Website Accessibility	Login page opens	Login page opened	PASS
TC02	Valid Login	Products page displayed	Products page displayed	PASS
TC03	Invalid Login	Error message displayed	Error message displayed	PASS
TC04	Product Display	Products displayed	Products displayed	PASS
TC05	Add to Cart	Product added	Product added	PASS
TC06	Remove from Cart	Product removed	Product removed	PASS
TC07	Shopping Cart	Selected product displayed	Product displayed	PASS
TC08	Checkout Information	Details accepted	Details accepted	PASS
TC09	Checkout Completion	Order completed	Order completed	PASS
TC10	Logout	Login page displayed	Login page displayed	PASS

Take a screenshot of the completed **Test Execution Report Excel sheet** showing all 10 test cases and their PASS/FAIL status.

Caption:

Figure 13: Test Execution Report

7. Test Execution Summary

Parameter	Result
Total Test Cases	10
Executed Test Cases	10
Passed	10

Failed	0
Blocked	0
Not Executed	0
Pass Percentage	100%
Fail Percentage	0%

Pass Percentage

8. Defect Report

Create another Excel sheet named:

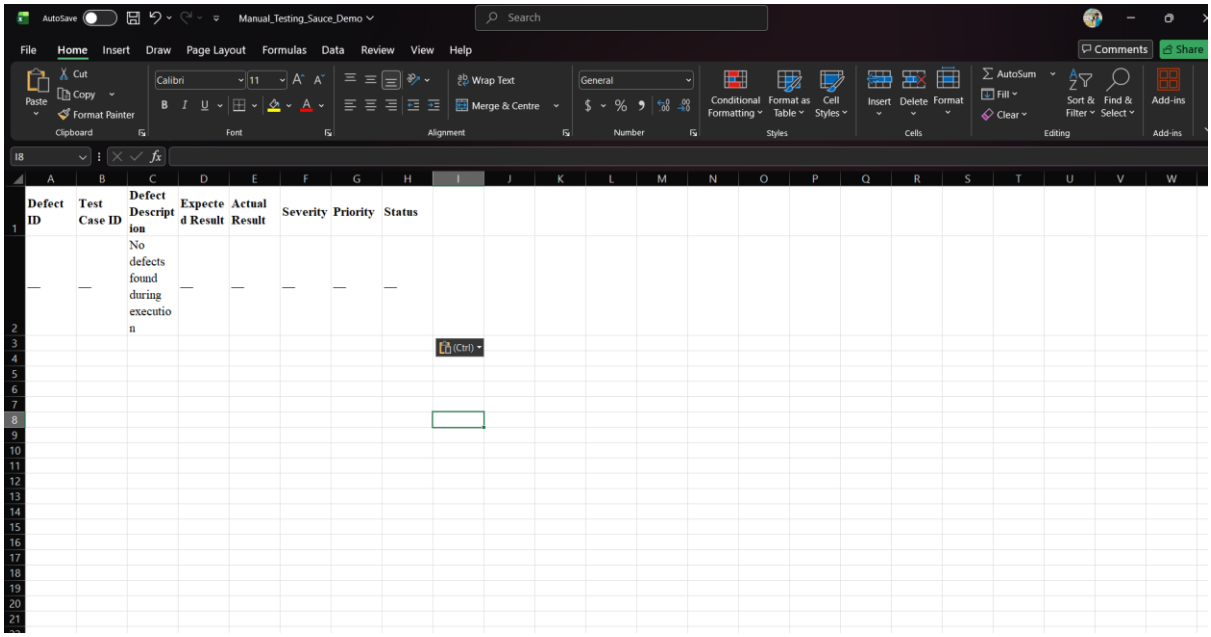
Defect Report

The defect report is used to record defects found during test execution.

Defect ID	Test Case ID	Defect Description	Expected Result	Actual Result	Severity	Priority	Status
—	—	No defects found during execution	—	—	—	—	—

If you actually find a problem while testing, enter it in this sheet instead.

Screenshot 14 – Defect Report



Take a screenshot of your completed **Defect Report** Excel sheet.

Caption:

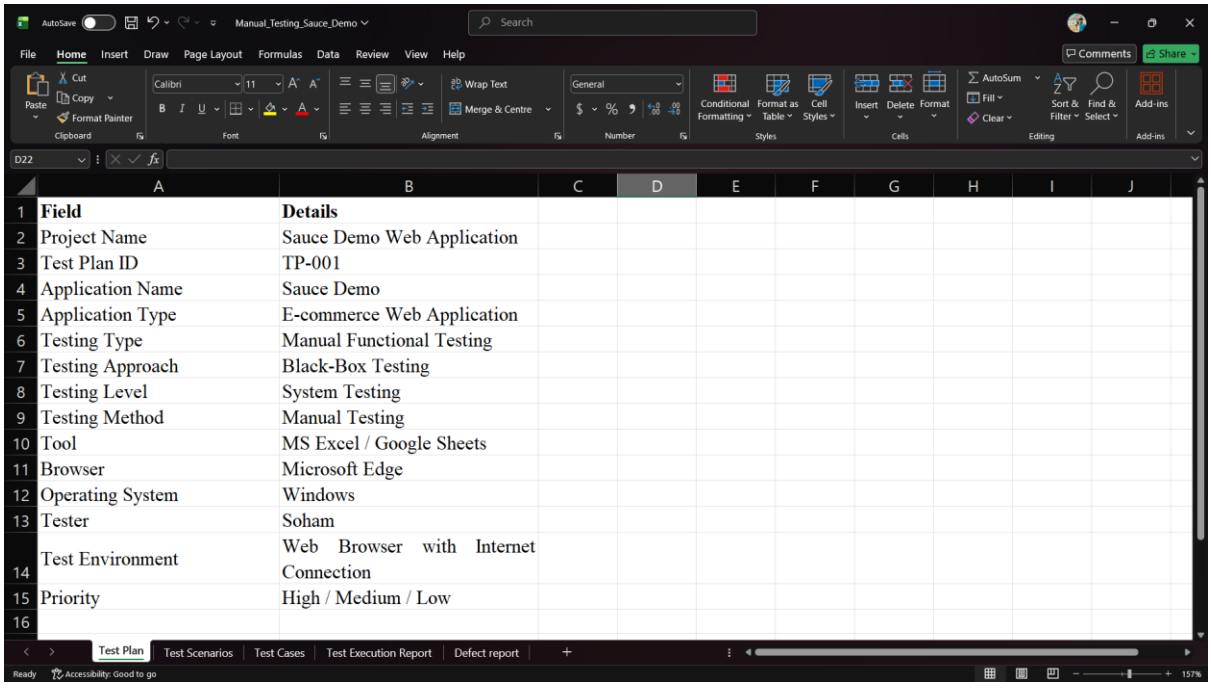
Figure 14: Defect Report

9. STLC Process

The practical follows the major phases of the **Software Testing Life Cycle**.

STLC Phase	Activity Performed
Requirement Analysis	Identified the functions of the Sauce Demo application
Test Planning	Prepared the Test Plan
Test Scenario Design	Identified testing scenarios
Test Case Design	Prepared detailed test cases
Test Environment Setup	Prepared browser and application environment
Test Execution	Executed test cases manually
Defect Reporting	Recorded defects, if any
Test Closure	Prepared execution summary and final report

Screenshot 15 – Complete Excel Workbook



Field	Details
Project Name	Sauce Demo Web Application
Test Plan ID	TP-001
Application Name	Sauce Demo
Application Type	E-commerce Web Application
Testing Type	Manual Functional Testing
Testing Approach	Black-Box Testing
Testing Level	System Testing
Testing Method	Manual Testing
Tool	MS Excel / Google Sheets
Browser	Microsoft Edge
Operating System	Windows
Tester	Soham
Test Environment	Web Browser with Internet Connection
Priority	High / Medium / Low

If possible, take **one screenshot showing the bottom of Excel with all five sheets visible**:

Test Plan | Test Scenarios | Test Cases | Test Execution Report | Defect Report

Caption:

Figure 15: Manual Testing Documentation Prepared in MS Excel

Practical No. 4

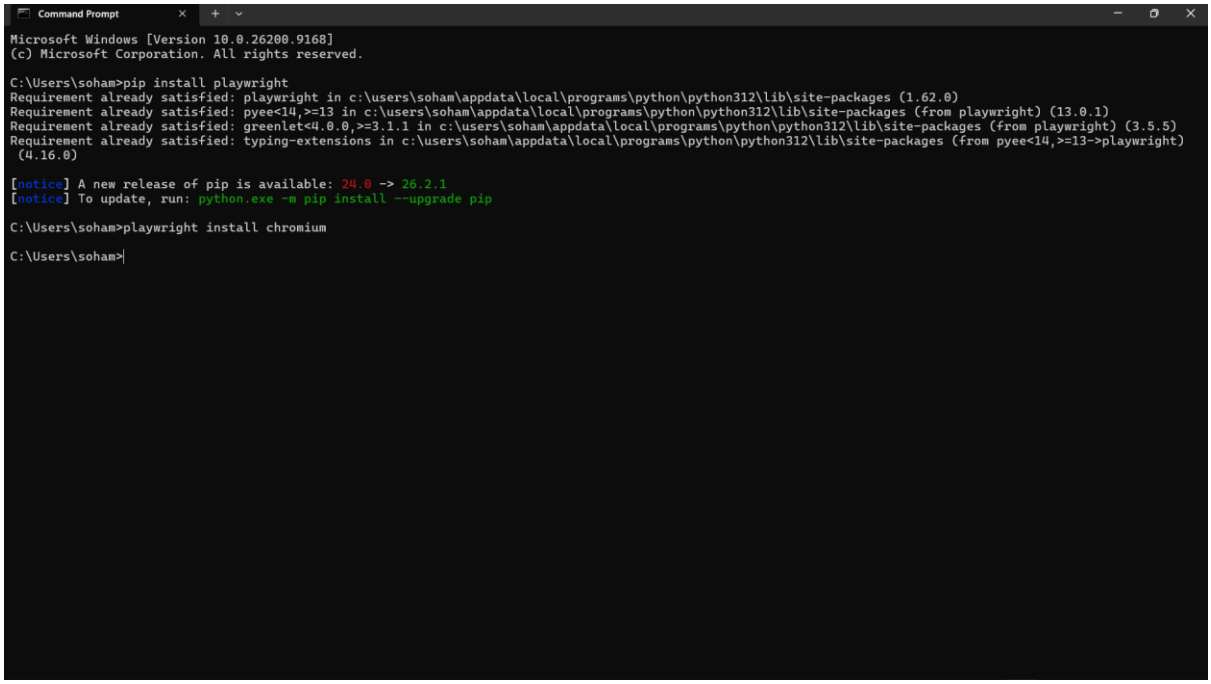
Web Automation: Login Functionality & Post-Login Assertion Verification

Aim

Write an automation script to perform login on a specified web page. Verify successful login using assertions such as URL validation, welcome message validation, and logout visibility check.

Steps:-

1. Open **Command Prompt (cmd)** and run the following commands one by one.
    a. **pip install playwright**
    b. **playwright install chromium**



2. Open Notepad and type this code and save file as **Practical4.py**

Code:-

```
from playwright.sync_api import sync_playwright, expect

def test_login_automation():
    with sync_playwright() as p:
        # Launch the official Google Chrome browser in visible mode
        browser = p.chromium.launch(channel="chrome", headless=False)
        context = browser.new_context()
        page = context.new_page()

        try:
```

```
        # 1. Navigate to the target login page (Using a standard practice site like
SauceDemo)
        print("Navigating to login page...")
        page.goto("https://www.saucedemo.com/")

        # 2. Perform Login Action
        print("Entering credentials...")
        page.fill("#user-name", "standard_user")
        page.fill("#password", "secret_sauce")
        page.click("#login-button")

        # 3. Validation: URL Check
        # Ensure the browser redirected to the inventory/dashboard page
        print("Validating URL...")
        expect(page).to_have_url("https://www.saucedemo.com/inventory.html")
        print("✓ URL Validation Passed: Successfully redirected.")

        # 4. Validation: Welcome Message / Dashboard Header Check
        # Check for a specific element unique to the logged-in dashboard
        print("Validating welcome/dashboard element...")
        header_element = page.locator(".title")
        expect(header_element).to_be_visible()
        expect(header_element).to_contain_text("Products")
        print("✓ Dashboard Header Validation Passed.")

        # 5. Validation: Logout Visibility Check
        # Open the side menu or locate the logout button to verify user session state
        print("Verifying logout button visibility...")
        # Open the menu (specific to this sample site, adapt IDs for your practical
page)
        page.click("#react-burger-menu-btn")

        logout_btn = page.locator("#logout_sidebar_link")
        expect(logout_btn).to_be_visible()
        print("✓ Logout Visibility Check Passed: Logout option is available.")

        print("\nAll login and validation tests completed successfully!")

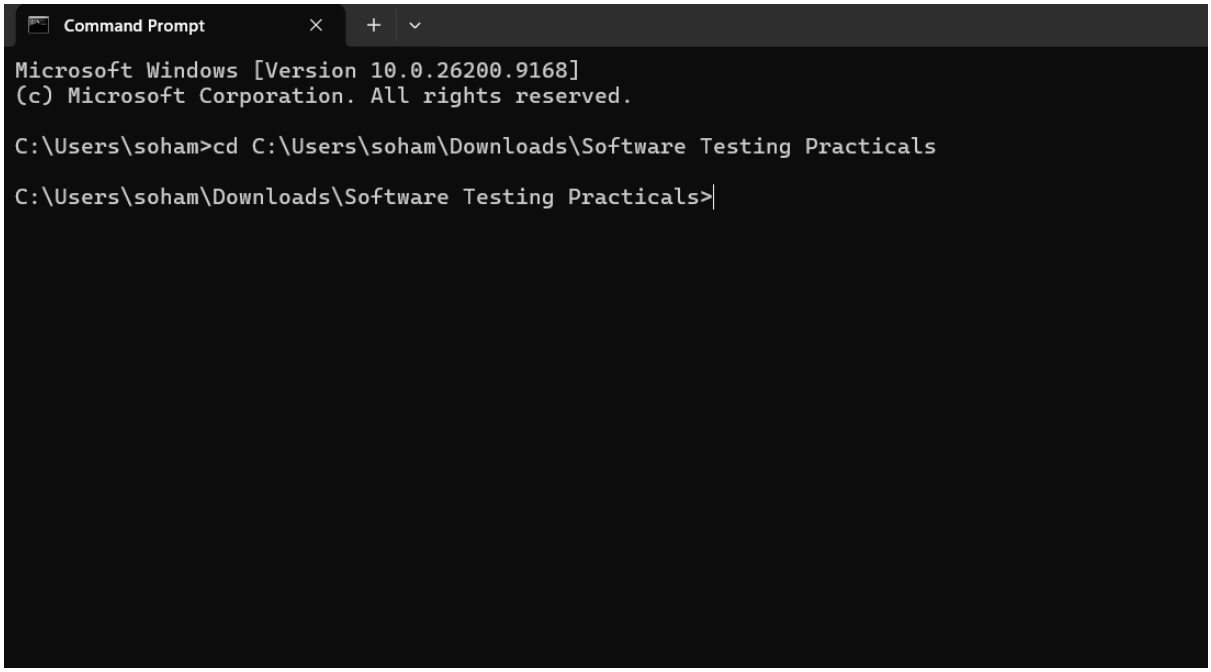
    except Exception as e:
        print(f"Test failed due to an error: {e}")

    finally:
        # Clean up and close the browser
```

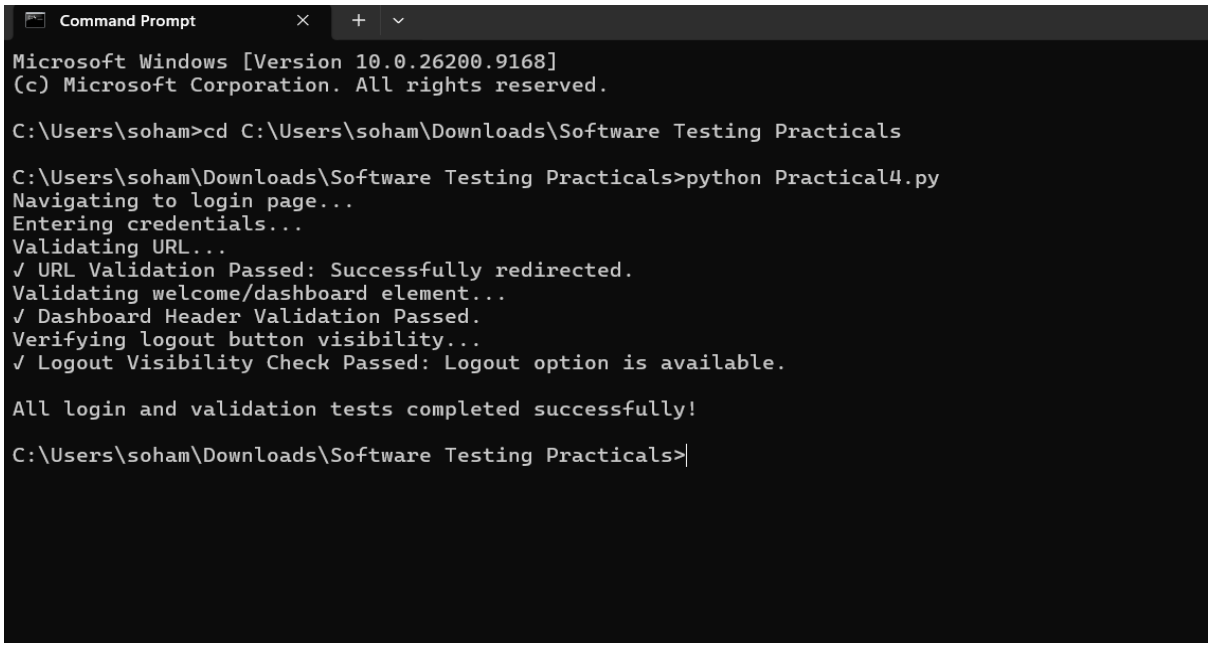
```
        page.wait_for_timeout(3000) # Brief pause to observe final state
        browser.close()

if __name__ == "__main__":
    test_login_automation()
```

3. Open **cmd** and navigate to the folder where you save **Practical4.py** by using **cd** command



4. Run the **Practical4.py** by the following command
    a. **python Practical4.py**



Practical No. 5

Web Automation: Element Interaction (TextBox, Radio, Checkbox, Dropdown, Alert, Frames) & Synchronization

Aim

Develop a script to automate interaction with various web elements (textbox, radio button, checkbox, dropdown, alert, and frames). Implement implicit and explicit waits for synchronization handling.

Steps:-

1. Open Notepad and type this code and save file as **Practical5.py**

   Code:-

```
from playwright.sync_api import sync_playwright, expect

def test_web_elements_and_synchronization():
    with sync_playwright() as p:
        browser = p.chromium.launch(channel="chrome", headless=False)
        context = browser.new_context()
        page = context.new_page()

        try:
            print("Navigating to test automation practice page...")
            page.goto("https://rahulshettyacademy.com/AutomationPractice/")

            # 1. SYNCHRONIZATION HANDLING
            page.set_default_timeout(5000)

            # 2. TEXTBOX INTERACTION
            print("Interacting with Textbox...")
            name_input = page.locator("#name")
            name_input.fill("Soham")
            expect(name_input).to_have_value("Soham")

            # 3. RADIO BUTTON INTERACTION
            print("Interacting with Radio Button...")
            radio_btn = page.locator("input[value='radio2']")
            radio_btn.click()
            expect(radio_btn).to_be_checked()

            # 4. CHECKBOX INTERACTION
```

```
            print("Interacting with Checkbox...")
            checkbox = page.locator("#checkBoxOption1")
            checkbox.check()
            expect(checkbox).to_be_checked()

            checkbox.uncheck()
            expect(checkbox).not_to_be_checked()

            # 5. DROPDOWN INTERACTION
            print("Interacting with Dropdown...")
            dropdown = page.locator("#dropdown-class-example")
            dropdown.select_option("option2")
            expect(dropdown).to_have_value("option2")

            # 6. ALERT HANDLING
            print("Handling JavaScript Alerts...")
            page.on("dialog", lambda dialog: dialog.accept())
            page.click("#alertbtn")

            # 7. FRAME HANDLING
            print("Handling Frames...")
            frame_element = page.frame_locator("#courses-iframe")
            inside_frame_heading = frame_element.locator("body")
            expect(inside_frame_heading).to_be_visible()
            print("✓ Successfully interacted with elements inside the frame.")

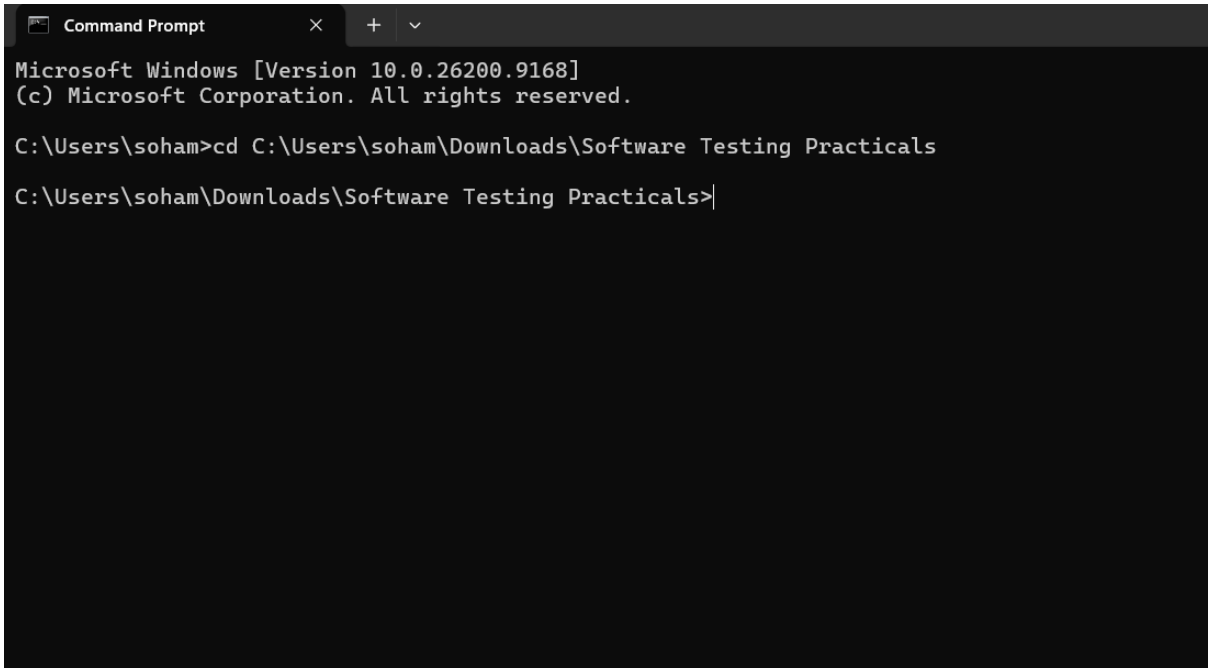
            print("\nAll web element interactions and synchronizations completed
successfully!")

        except Exception as e:
            print(f"Test encountered an error: {e}")

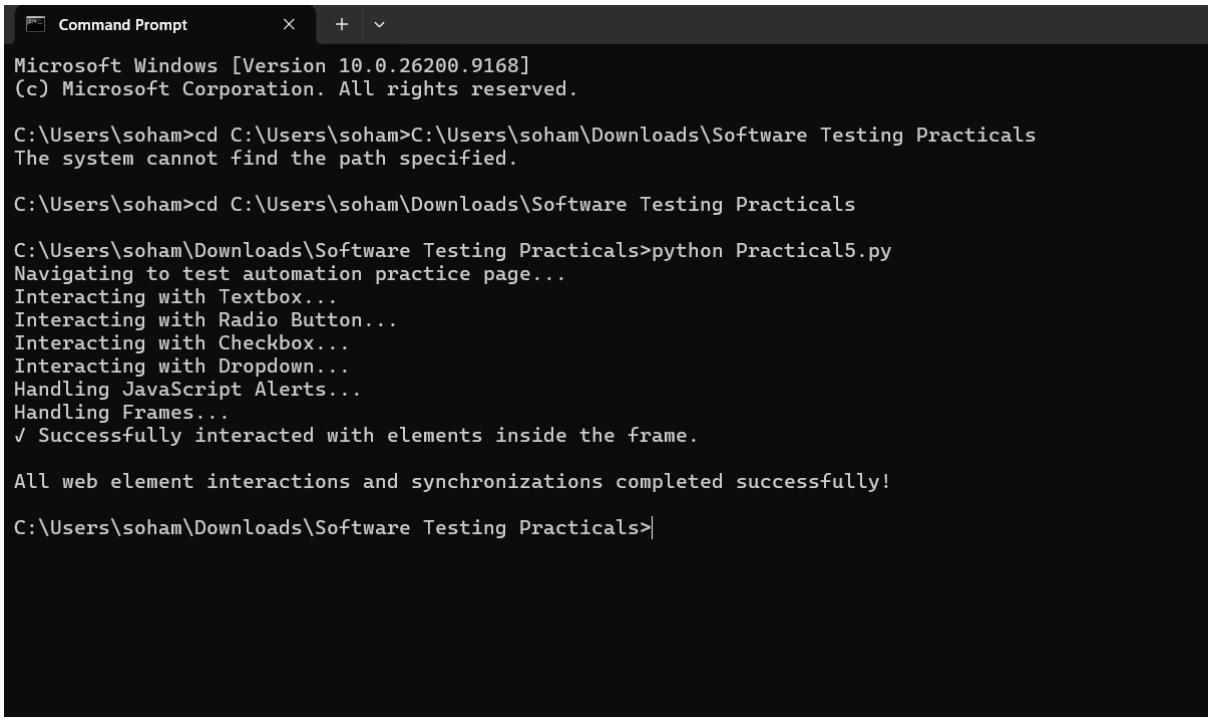
        finally:
            page.wait_for_timeout(3000)
            browser.close()

if __name__ == "__main__":
    test_web_elements_and_synchronization()
```

2. Open **cmd** and navigate to the folder where you save **Practical5.py** by using **cd** command



3. Run the **Practical5.py** by the following command
    a. **python Practical5.py**



Practical No. 6

Web Automation: Dynamic Table Data Extraction & Conditional Analytics

Aim

Automate extraction of student marks from a dynamic web table and determine the number of students scoring above 60 in at least one subject. Compute total count and percentage.

Steps:-

1. Open Notepad and type this code and save file as **Practical6.py**

   Code:-

```
from playwright.sync_api import sync_playwright

def test_web_table_score_analysis():
    with sync_playwright() as p:
        browser = p.chromium.launch(channel="chrome", headless=False)
        context = browser.new_context()
        page = context.new_page()

        try:
            print("Navigating to table test page...")
            page.goto("https://www.w3schools.com/html/html_tables.asp")

            print("Extracting data from the web table...")
            table_rows = page.locator("table#customers tr")
            row_count = table_rows.count()

            print(f"Total rows found in table: {row_count}")

            total_students = 0
            scoring_above_60_count = 0

            for i in range(1, row_count):
                row = table_rows.nth(i)
                columns = row.locator("td")

                if columns.count() > 0:
                    total_students += 1

                    has_scored_above_60 = False
```

```
                for j in range(1, columns.count()):
                    cell_text = columns.nth(j).inner_text().strip()

                    try:
                        marks = float(cell_text)
                        if marks > 60:
                            has_scored_above_60 = True
                            break
                    except ValueError:
                        continue

                if has_scored_above_60:
                    scoring_above_60_count += 1

            print("\n--- Score Analysis Report ---")
            print(f"Total Students Evaluated: {total_students}")
            print(f"Students Scoring Above 60 in at least one subject:
{scoring_above_60_count}")

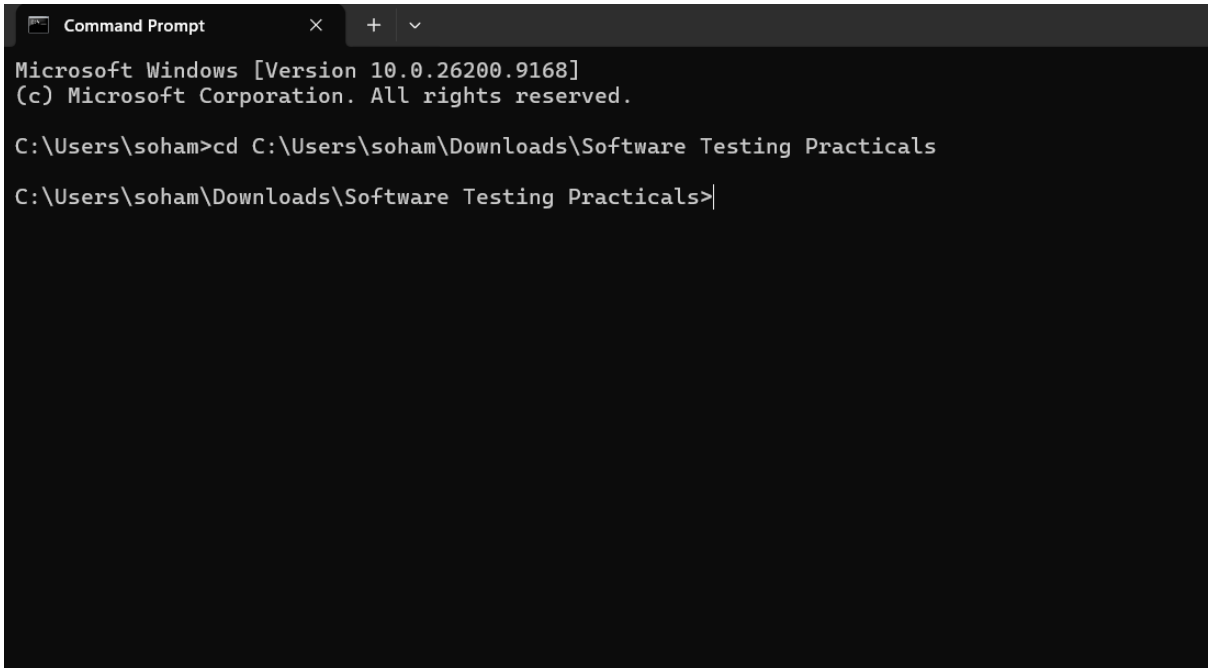
            if total_students > 0:
                percentage = (scoring_above_60_count / total_students) * 100
                print(f"Percentage: {percentage:.2f}%")
            else:
                print("Percentage: 0.00% (No student records found)")

        except Exception as e:
            print(f"Test encountered an error: {e}")

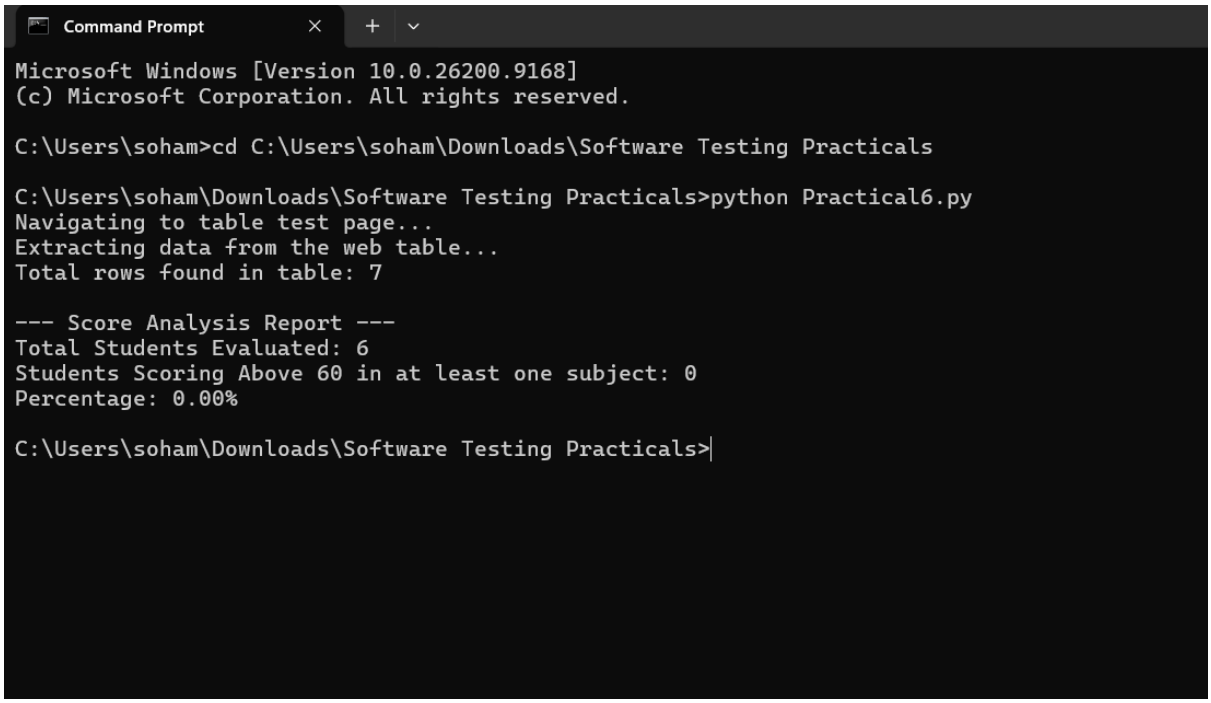
        finally:
            page.wait_for_timeout(4000)
            browser.close()

if __name__ == "__main__":
    test_web_table_score_analysis()
```

2. Open **cmd** and navigate to the folder where you save **Practical6.py** by using **cd** command



3. Run the **Practical6.py** by the following command
    a. **python Practical6.py**



Practical No. 7

Web Automation: Element Enumeration & Categorized Component Count

Aim

Write a program to identify and count the total number of objects present on a web page including links, images, buttons, and input elements. Display categorized counts.

Steps:-

1. Open Notepad and type this code and save file as **Practical7.py**

   Code:-

```
from playwright.sync_api import sync_playwright

def test_object_counting():
    with sync_playwright() as p:
        browser = p.chromium.launch(channel="chrome", headless=False)
        context = browser.new_context()
        page = context.new_page()

        try:
            print("Navigating to target web page...")
            page.goto("https://rahulshettyacademy.com/AutomationPractice/")

            print("Identifying and counting web elements...")

            links = page.locator("a")
            link_count = links.count()

            images = page.locator("img")
            image_count = images.count()

            buttons = page.locator("button, input[type='button'], input[type='submit']")
            button_count = buttons.count()

            inputs =
page.locator("input:not([type='hidden']):not([type='checkbox']):not([type='radio']):not
([type='button']):not([type='submit'])")
            input_count = inputs.count()

            total_elements = page.locator("*").count()
```

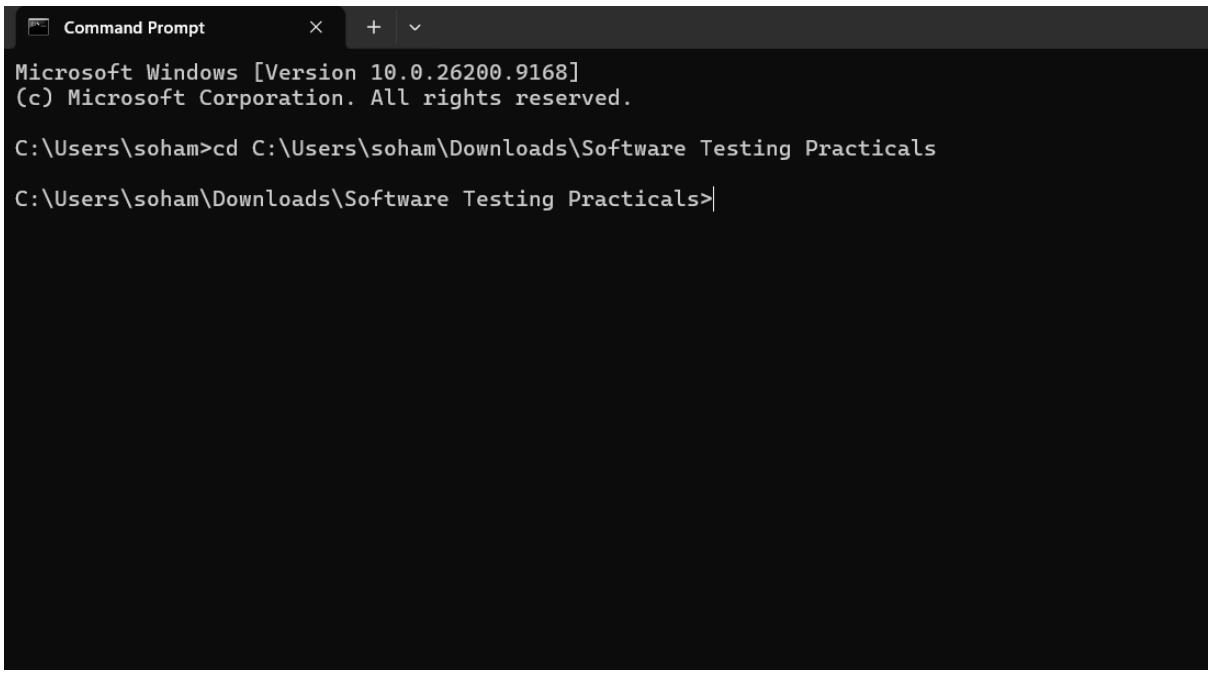
```
        print("\n" + "="*40)
        print("      WEB OBJECT COUNTING REPORT")
        print("="*40)
        print(f"  Total Links (<a>)              : {link_count}")
        print(f"  Total Images (<img>)           : {image_count}")
        print(f"  Total Buttons                  : {button_count}")
        print(f"  Total Input Textboxes          : {input_count}")
        print("-" * 40)
        print(f"  Total DOM Elements on Page     : {total_elements}")
        print("="*40)

    except Exception as e:
        print(f"Test encountered an error: {e}")

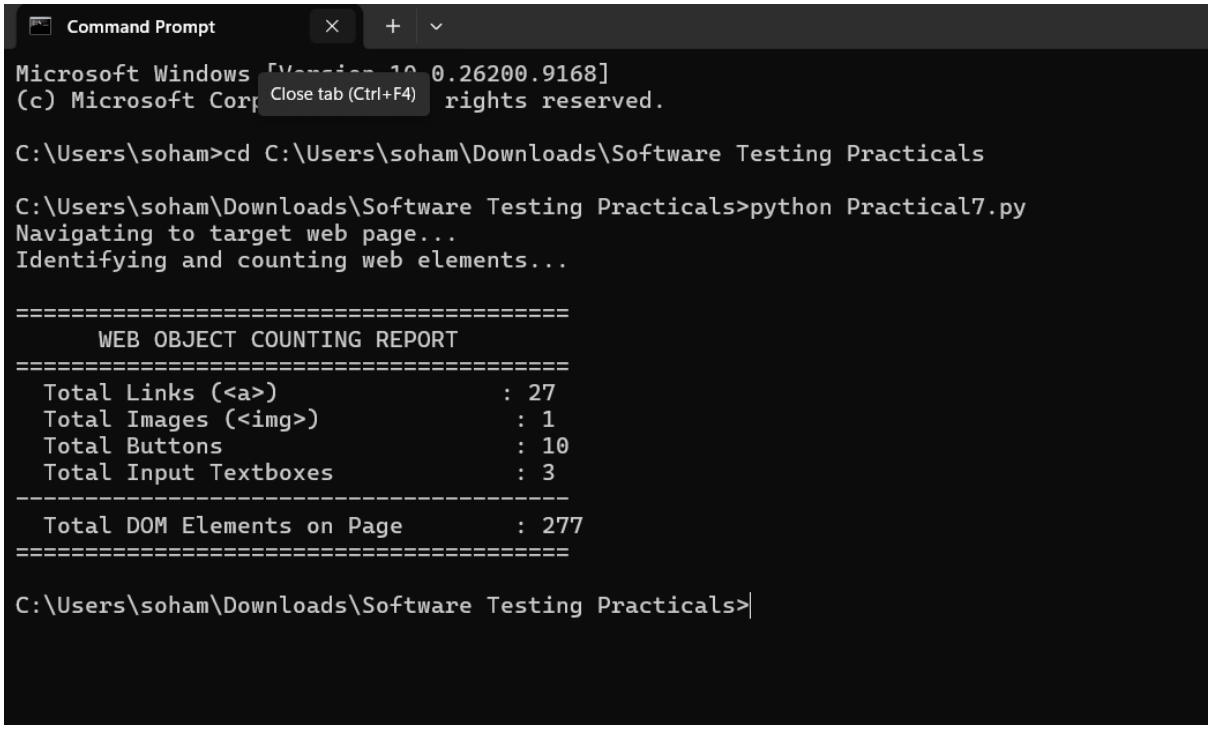
    finally:
        page.wait_for_timeout(3000)
        browser.close()

if __name__ == "__main__":
    test_object_counting()
```

2. Open **cmd** and navigate to the folder where you save **Practical7.py** by using **cd** command



3. Run the **Practical7.py** by the following command
    a. **python Practical7.py**



Practical No. 8

Web Automation: Dropdown Enumeration, Default Value Validation & Selection Testing

Aim

Automate the process of identifying a list or dropdown box on a web page and determine the total number of items. Validate default selected value and perform selection testing.

Steps:-

1. Open Notepad and type this code and save file as **Practical8.py**

   Code:-

```
from playwright.sync_api import sync_playwright, expect

def test_dropdown_and_combo_verification():
    with sync_playwright() as p:
        browser = p.chromium.launch(channel="chrome", headless=False)
        context = browser.new_context()
        page = context.new_page()
        try:
            print("Navigating to test automation practice page...")
            page.goto("https://rahulshettyacademy.com/AutomationPractice/")

            dropdown_locator = page.locator("#dropdown-class-example")

            # 1. TOTAL NUMBER OF ITEMS DETERMINATION
            print("Counting total items in the dropdown...")
            options_locator = dropdown_locator.locator("option")
            total_items = options_locator.count()
            print(f"? Total items found in dropdown: {total_items}")

            for i in range(total_items):
                print(f"  - Item {i}: {options_locator.nth(i).inner_text()} (Value:
{options_locator.nth(i).get_attribute('value')})")

            # 2. DEFAULT SELECTED VALUE VALIDATION
            print("\nValidating default selected value...")
            default_value = dropdown_locator.input_value()
            print(f"Default selected value attribute: '{default_value}'")

            expect(dropdown_locator).to_have_value("")
```

```
            print("? Default selected value validation passed.")

            # 3. SELECTION TESTING (Selecting by Value, Label, and Index)
            print("\nPerforming selection testing...")

            print("Selecting option by visible text: 'Option2'...")
            dropdown_locator.select_option(label="Option2")
            expect(dropdown_locator).to_have_value("option2")
            print("? Selection by Label passed.")

            print("Selecting option by value: 'option3'...")
            dropdown_locator.select_option(value="option3")
            expect(dropdown_locator).to_have_value("option3")
            print("? Selection by Value passed.")

            print("Selecting option by index: 1 (First actual choice)...")
            dropdown_locator.select_option(index=1)
            print("? Selection by Index passed.")

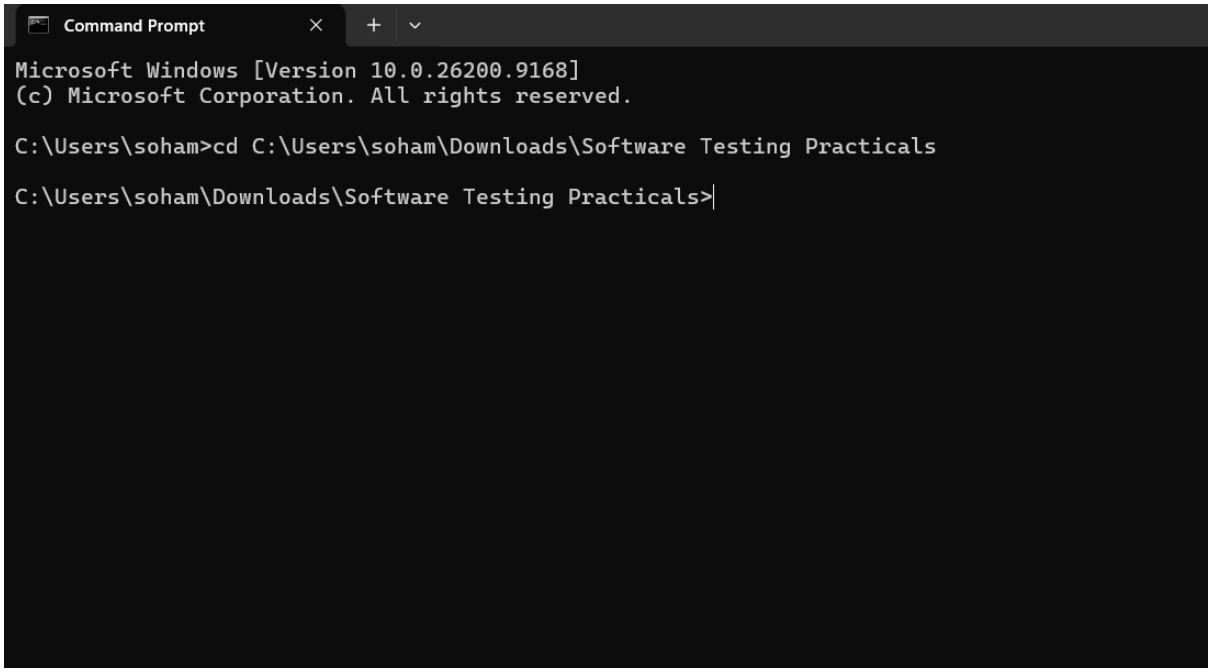
            print("\nAll list and combo box verifications completed successfully!")

        except Exception as e:
            print(f"Test encountered an error: {e}")

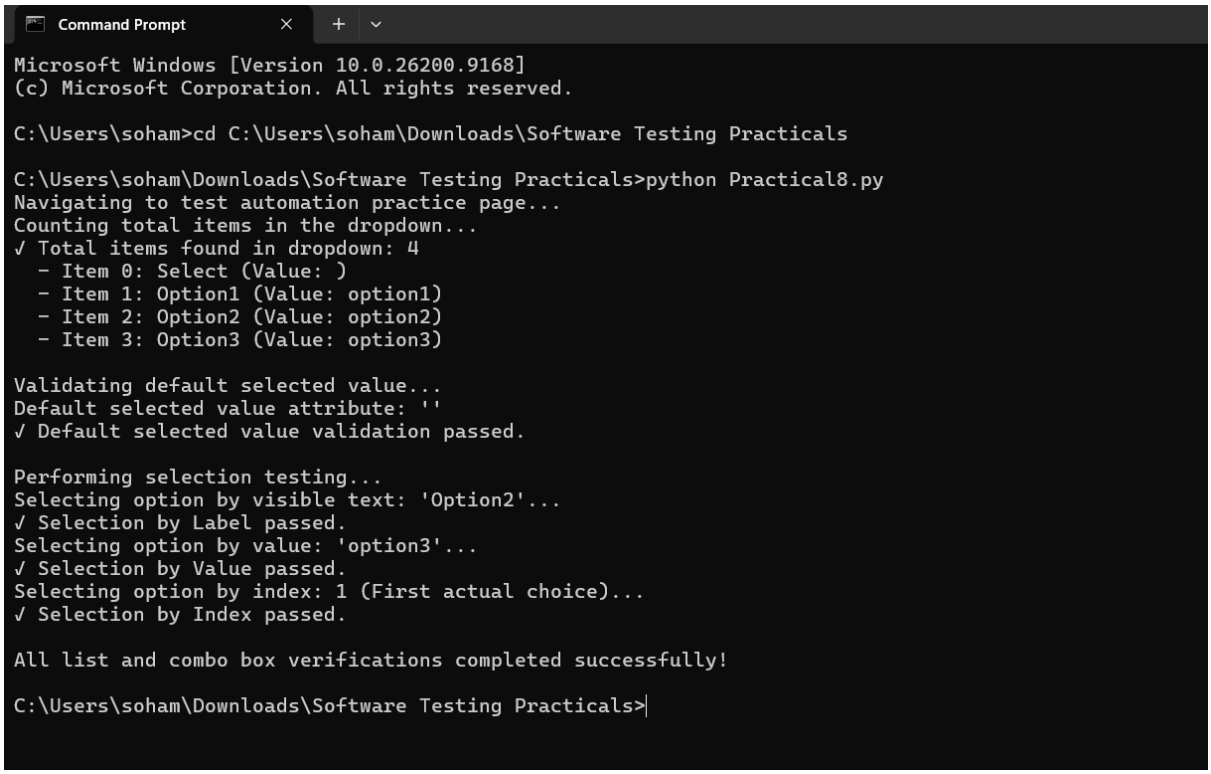
        finally:
            page.wait_for_timeout(3000)
            browser.close()

if __name__ == "__main__":
    test_dropdown_and_combo_verification()
```

2. Open **cmd** and navigate to the folder where you save **Practical8.py** by using **cd** command



3. Run the **Practical8.py** by the following command
    a. **python Pracctical8.py**



Practical No. 9

Web Automation: Checkbox Enumeration, Status Counting & Dynamic Selection Validation

Aim

Write a program to count the total number of checkboxes on a web page, including checked and unchecked counts. Perform dynamic selection and validation.

Steps:-

1. Open Notepad and type this code and save file as **Practical9.py**

   Code:-

```
from playwright.sync_api import sync_playwright, expect

def test_checkbox_identification_and_validation():
    with sync_playwright() as p:
        browser = p.chromium.launch(channel="chrome", headless=False)
        context = browser.new_context()
        page = context.new_page()

        try:
            print("Navigating to test automation practice page...")
            page.goto("https://rahulshettyacademy.com/AutomationPractice/")

            # 1. IDENTIFY AND COUNT ALL CHECKBOXES
            print("Scanning for checkboxes on the page...")
            checkboxes = page.locator("input[type='checkbox']")
            total_checkboxes = checkboxes.count()
            print(f"✓ Total Checkboxes Found: {total_checkboxes}")

            # 2. COUNT CHECKED AND UNCHECKED CHECKBOXES
DYNAMICALLY
            checked_count = 0
            unchecked_count = 0

            for i in range(total_checkboxes):
                cb = checkboxes.nth(i)
                if cb.is_checked():
                    checked_count += 1
                else:
```

```
                unchecked_count += 1

            print(f"  - Currently Checked Checkboxes    : {checked_count}")
            print(f"  - Currently Unchecked Checkboxes : {unchecked_count}")

            # 3. PERFORM DYNAMIC SELECTION AND VALIDATION
            print("\nPerforming dynamic selection testing...")

            target_checkbox = page.locator("#checkBoxOption1")

            if not target_checkbox.is_checked():
                print("Checking 'checkBoxOption1'...")
                target_checkbox.check()

                expect(target_checkbox).to_be_checked()
                print("✓ Validation Passed: Checkbox is successfully checked.")

            print("Unchecking 'checkBoxOption1'...")
            target_checkbox.uncheck()

            expect(target_checkbox).not_to_be_checked()
            print("✓ Validation Passed: Checkbox is successfully unchecked.")

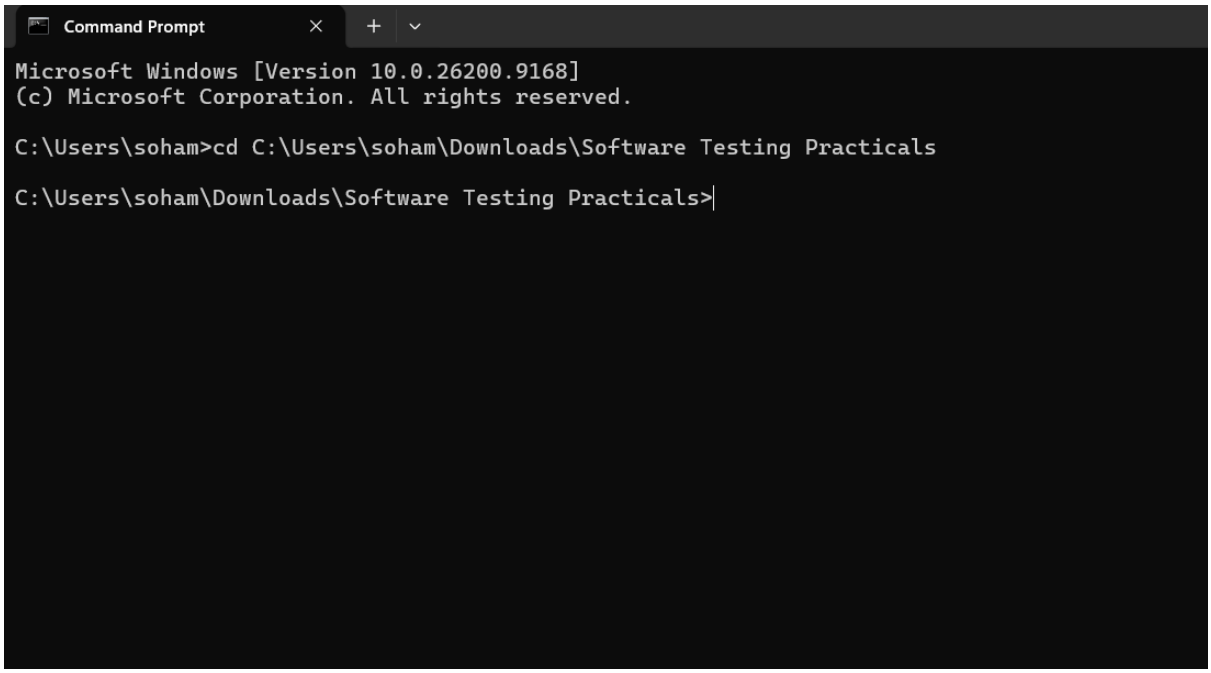
            print("\nAll checkbox identification and validation tests completed
successfully!")

        except Exception as e:
            print(f"Test encountered an error: {e}")

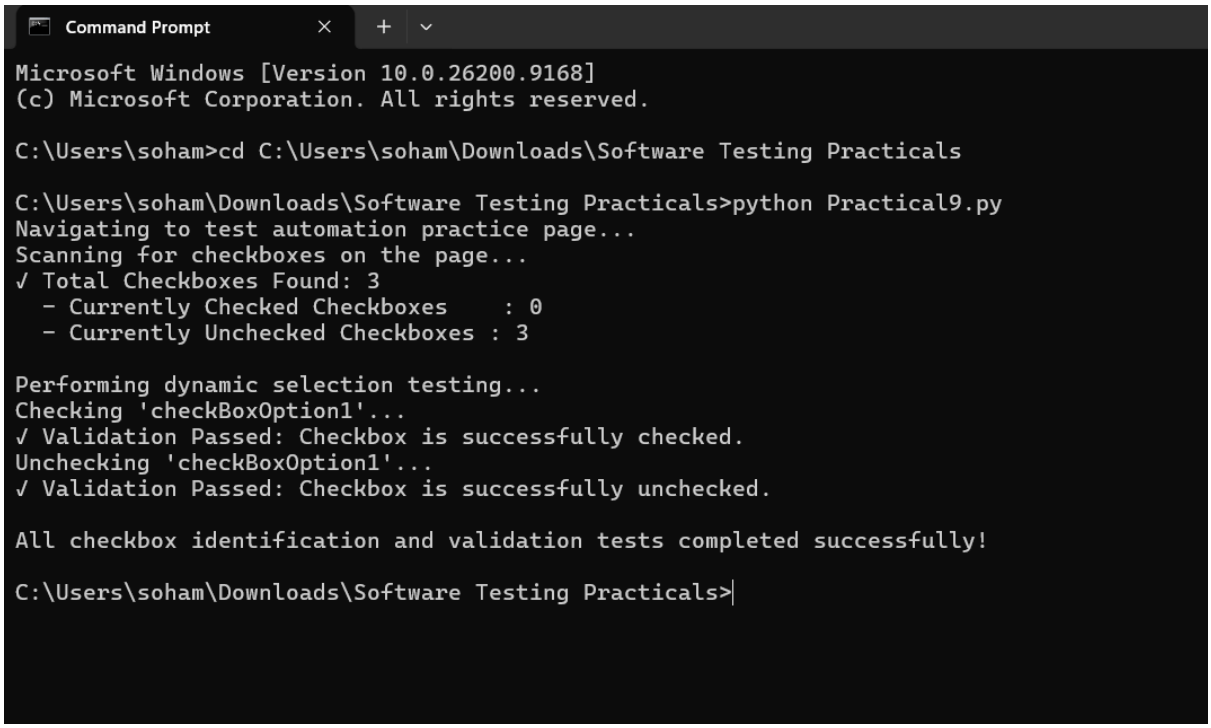
        finally:
            page.wait_for_timeout(3000)
            browser.close()

if __name__ == "__main__":
    test_checkbox_identification_and_validation()
```

2. Open **cmd** and navigate to the folder where you save **Practical9.py** by using **cd** command



3. Run the **Practical9.py** by the following command
    a. **python Practical9.py**



Practical No. 10

Web Automation: Dynamic Table Data Extraction, Sorting Validation & Filter Verification

Aim

Automate extraction of data from a dynamic web table. Validate sorting functionality and verify correctness of filtered results.

Steps:-

1. Open Notepad and type this code and save file as **Practical10.py**

   Code:-

```
from playwright.sync_api import sync_playwright, expect

def test_dynamic_table_sorting():
    with sync_playwright() as p:
        browser = p.chromium.launch(channel="chrome", headless=False)
        page = browser.new_page()

        try:
            page.goto("https://rahulshettyacademy.com/seleniumPractise/#/offers")

            name_column_locator = page.locator("tbody tr td:nth-child(1)")

            before_sort_list = name_column_locator.all_text_contents()
            print(f"Items before sort: {before_sort_list[:5]}...")

            print("Triggering sort functionality...")
            page.click("th[aria-label*='Veg/fruit name']")

            page.wait_for_load_state("networkidle")

            after_sort_list = name_column_locator.all_text_contents()
            print(f"Items after sort:  {after_sort_list[:5]}...")

            # Fix: Verify if the after-sort list is properly sorted alphabetically (case-insensitive)
            expected_sorted_list = sorted(after_sort_list, key=lambda s: s.lower())

            print("\nValidating sorting logic...")
            if after_sort_list == expected_sorted_list:
```

```
            print("✓ Sorting Validation Passed: The table is correctly sorted
alphabetically.")
        else:
            print("✗ Sorting Validation Failed: Data does not match expected order.")

        print("\nTesting filter functionality...")
        search_input = page.locator("#search-field")
        search_term = "Banana"
        search_input.fill(search_term)

        for row in name_column_locator.all():
            expect(row).to_contain_text(search_term)

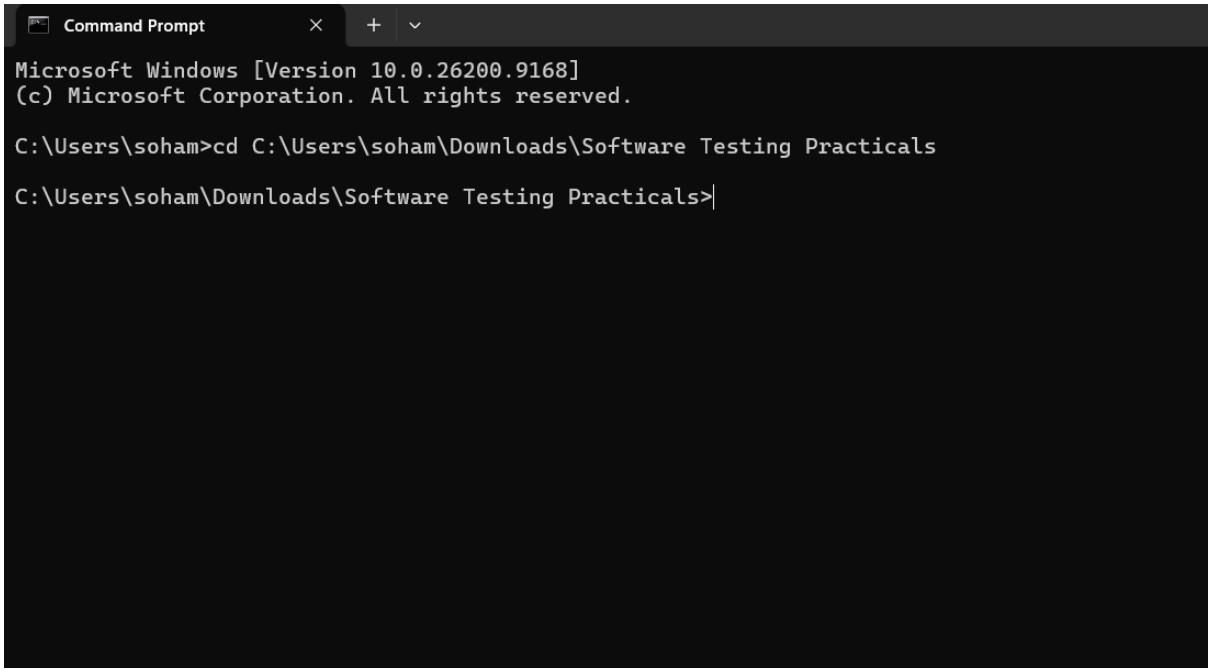
        print(f"✓ Filter Validation Passed: All visible rows contain '{search_term}'.")

    except Exception as e:
        print(f"Test encountered an error: {e}")

    finally:
        page.wait_for_timeout(3000)
        browser.close()

if __name__ == "__main__":
    test_dynamic_table_sorting()
```

2. Open **cmd** and navigate to the folder where you save **Practical10.py** by using **cd** command



3. Run the **Practical10.py** by the following command
    a. **python Pracctical10.py**

